



Analizador inteligente de ocupación en bandas ISM mediante sistema digital con memoria direccionada y modelo TinyML para selección adaptativa de canal



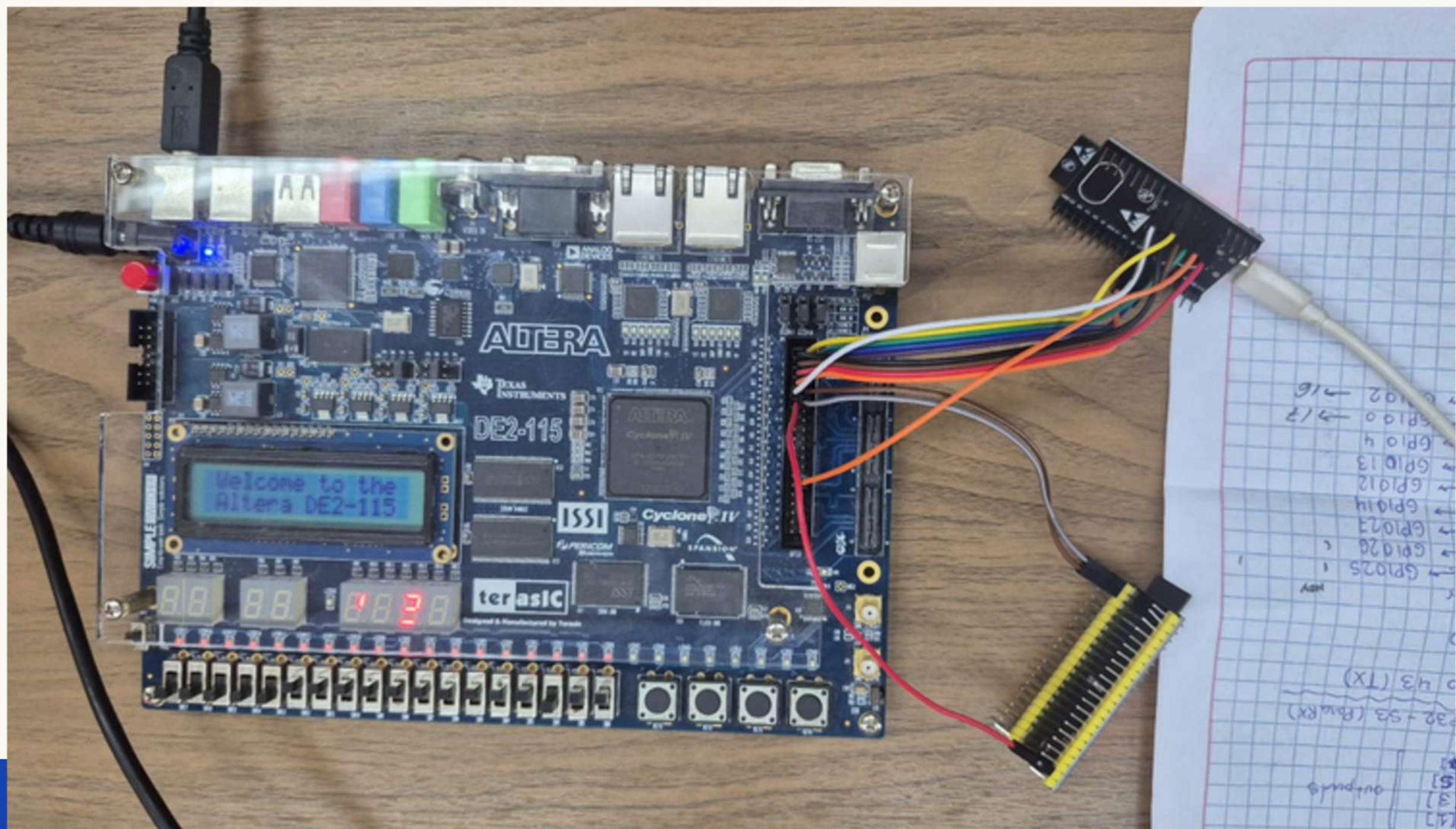
Entregable 5

Docente

- Villafuerte Barreto, Hernán Oswaldo

Integrantes: Grupo 1

- Agelvis Vargas, Gustavo Enrique
- Espinoza Pagola, Frank Abner
- Izarra Gamarra, Annabeth Valeria
- Vallejos Terrones, Brenda Lucero



Problemática



El crecimiento acelerado de dispositivos inalámbricos en entornos industriales, urbanos y académicos **ha saturado la banda ISM**, afectando directamente el rendimiento, la confiabilidad y la experiencia de los usuarios.



¿QUÉ PROVOCA ESTA SATURACIÓN?



Interferencias

Señales que se superponen y distorsionan otras transmisiones.



Aumento de latencia

Respuestas más lentas que degradan aplicaciones en tiempo real.



Pérdida de paquetes

Datos que no llegan a su destino, generando retransmisiones y ineficiencia.



Disminución de la calidad del servicio (QoS)

Menor velocidad, cortes, inestabilidad y mala experiencia.



REALIDAD EN LIMA

En universidades, industrias, oficinas y zonas urbanas, la banda ISM de **2.4 GHz** está cada vez más **congestionada**:



Para **cientos de dispositivos** conectados al mismo tiempo.



Compartiendo un número **limitado de canales** (solo **3 canales** no superpuestos).



En entornos donde la **demanda de conectividad** **crece día a día**.



- ✗ CORTES
- ✗ LENTITUD
- ✗ INESTABILIDAD
- ✗ PÉRDIDA DE DATOS
- ✗ USUARIOS INSATISFECHOS

¡NO PODEMOS SEGUIR ASÍ!



SE REQUIERE UN SISTEMA INTELIGENTE QUE PERMITA:

Monitorear • Analizar • Detectar • Decidir • Actuar
en tiempo real ante distintas situaciones de congestión.



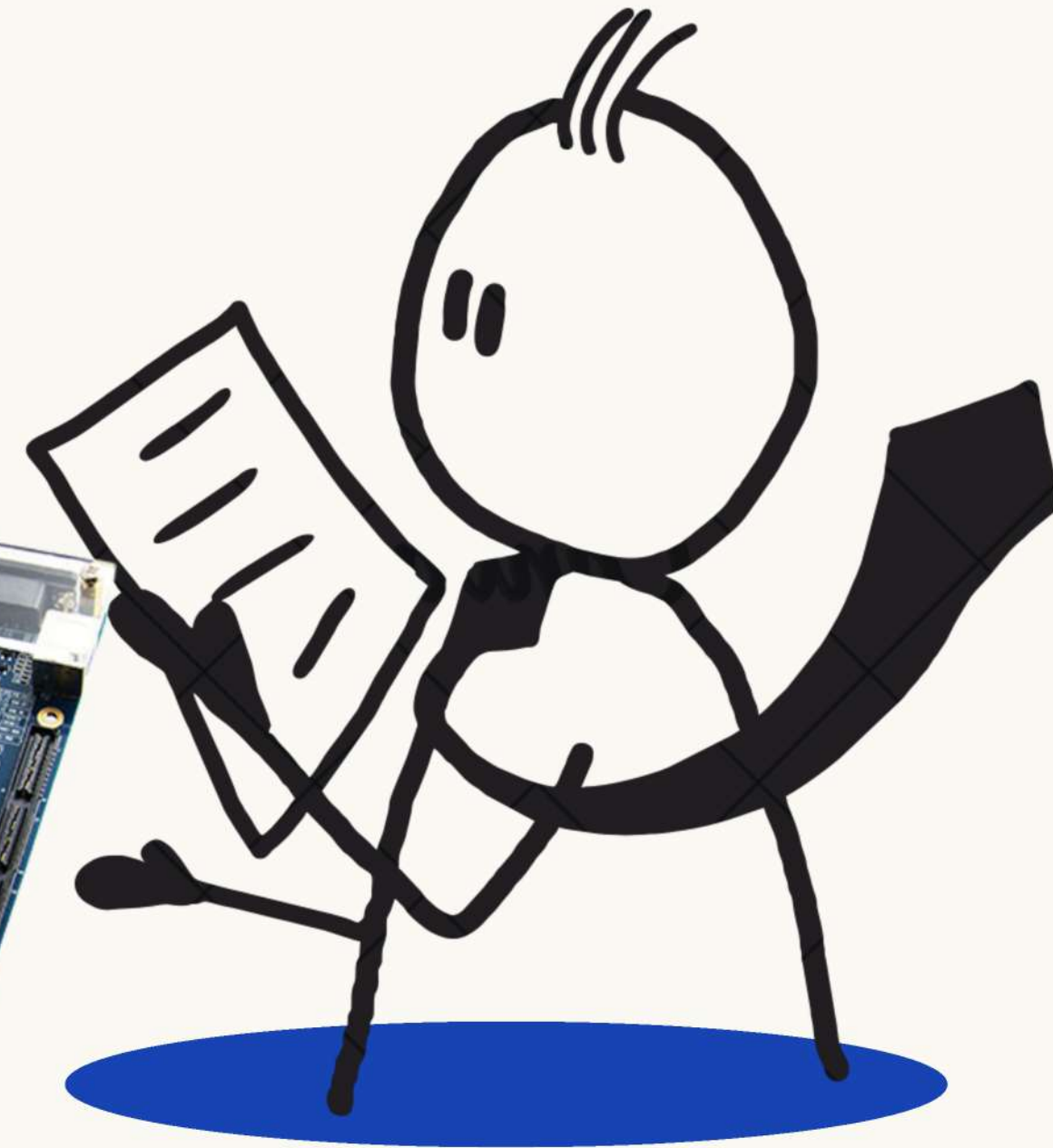
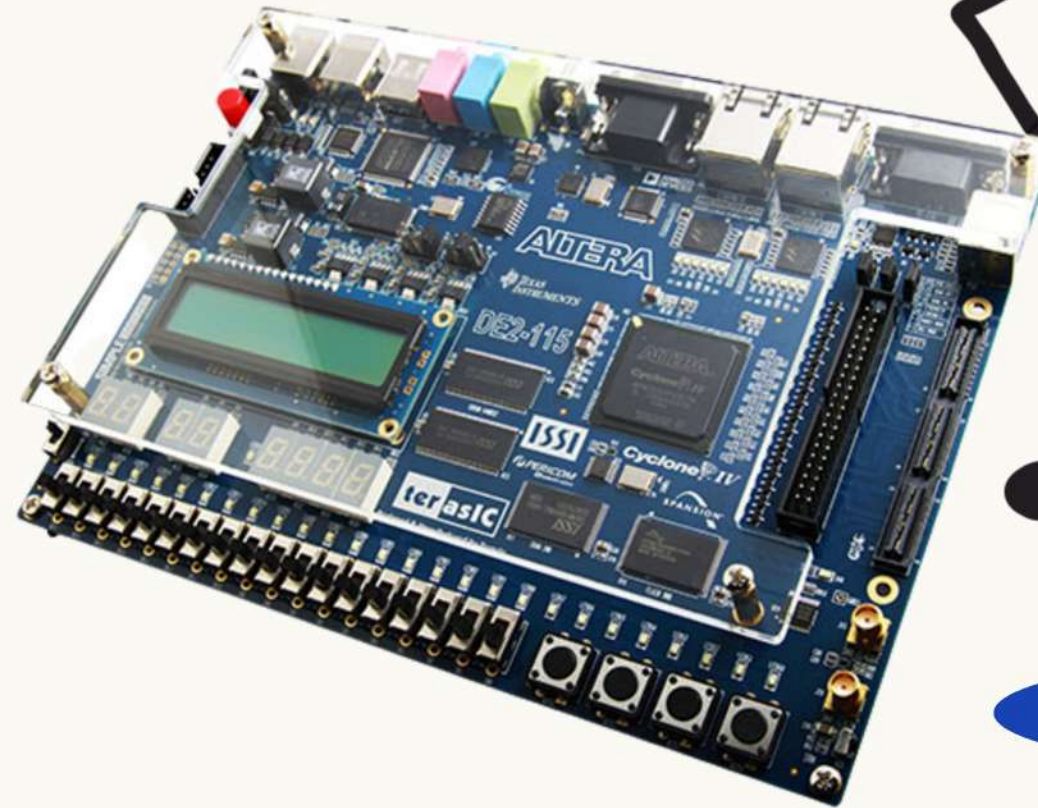
La solución está en la **INTELIGENCIA** para entender el entorno y actuar antes de que el problema afecte al usuario.

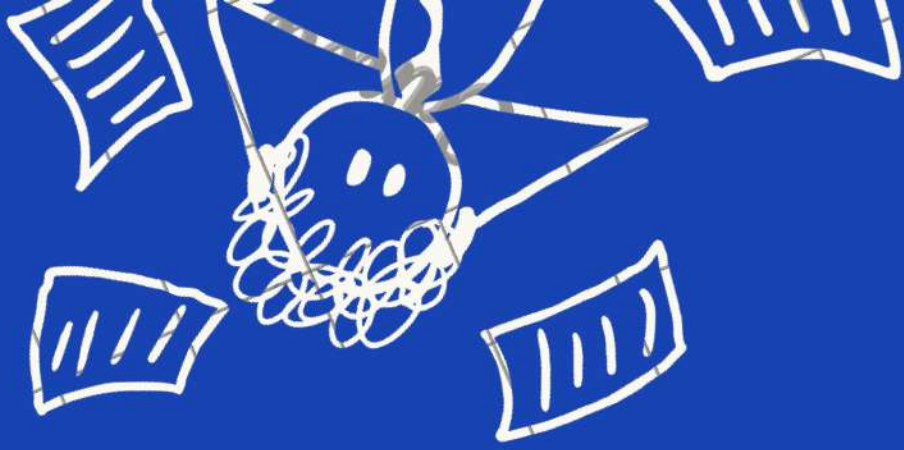
Presentación del Proyecto

El proyecto implementa un sistema digital capaz de analizar métricas de ocupación de canales WiFi en la banda ISM, almacenar información en memoria direccionada y seleccionar automáticamente el canal menos congestionado mediante una FSM y lógica de decisión LUT implementadas en FPGA.

Materiales

- FPGA DE2-115 (Cyclone IV E)
- ESP32-S3
- Laptop con Quartus Prime y ModelSim
- Cable USB-Blaster





Sobre la situación actual

La situación está en que muchas tecnologías no plantean un mecanismo local tanto económico como autónomo que pueda analizar la actividad de distintas bandas

Este dispositivo se basaría en parámetros como calidad de transmisión, potencia, entre otros para poder tomar decisiones digitales que mejoren el desempeño del enlace. Lo cual, cuando se observa el sector IoT notamos que muchos utilizan configuraciones estáticas sin considerar lo mencionado anteriormente.

Fuente

Abdelghany, A., Uguen, B., Moy, C., & Lemur, D. (2022). Decentralized Adaptive Spectrum Learning in Wireless IoT Networks Based on Channel Quality Information.

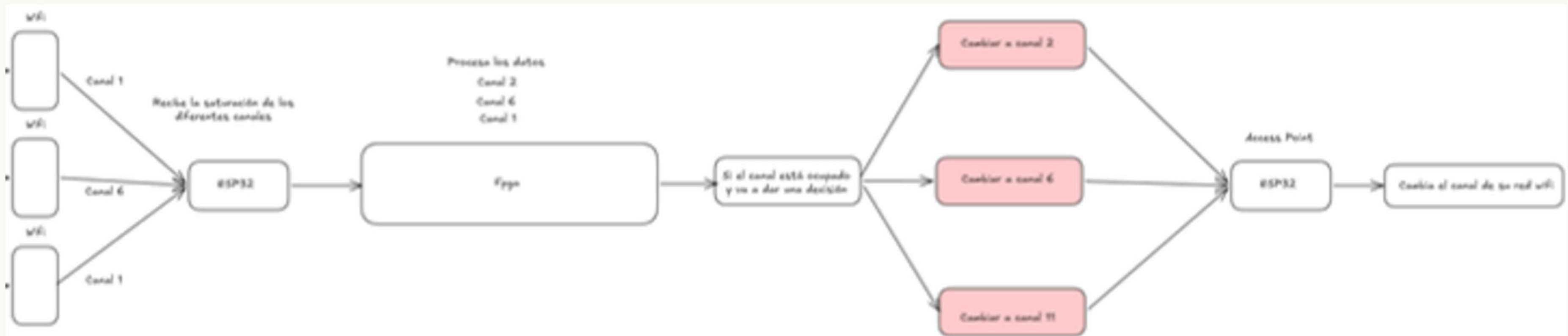
Casos de uso

En un entorno industrial con múltiples dispositivos IoT, redes WiFi, sensores NB-IoT, cámaras y sistemas de automatización transmitiendo simultáneamente en la banda ISM, puede producirse congestión e interferencia que afecte la estabilidad y latencia de las comunicaciones críticas.

En este escenario, el sistema propuesto monitorea continuamente la ocupación de los canales inalámbricos mediante el ESP32-S3 y envía las métricas hacia la FPGA, donde son procesadas en tiempo real mediante FSM, memoria direccionada y lógica de decisión LUT para identificar el canal menos congestionado. Esto permite recomendar o ejecutar cambios dinámicos de canal para mantener una comunicación más estable, reducir pérdida de paquetes y mejorar la confiabilidad de sistemas industriales conectados.



De forma general la propuesta de proyecto



Diseño de la solución

Diagrama de bloques

FPGA

ESP32-S3

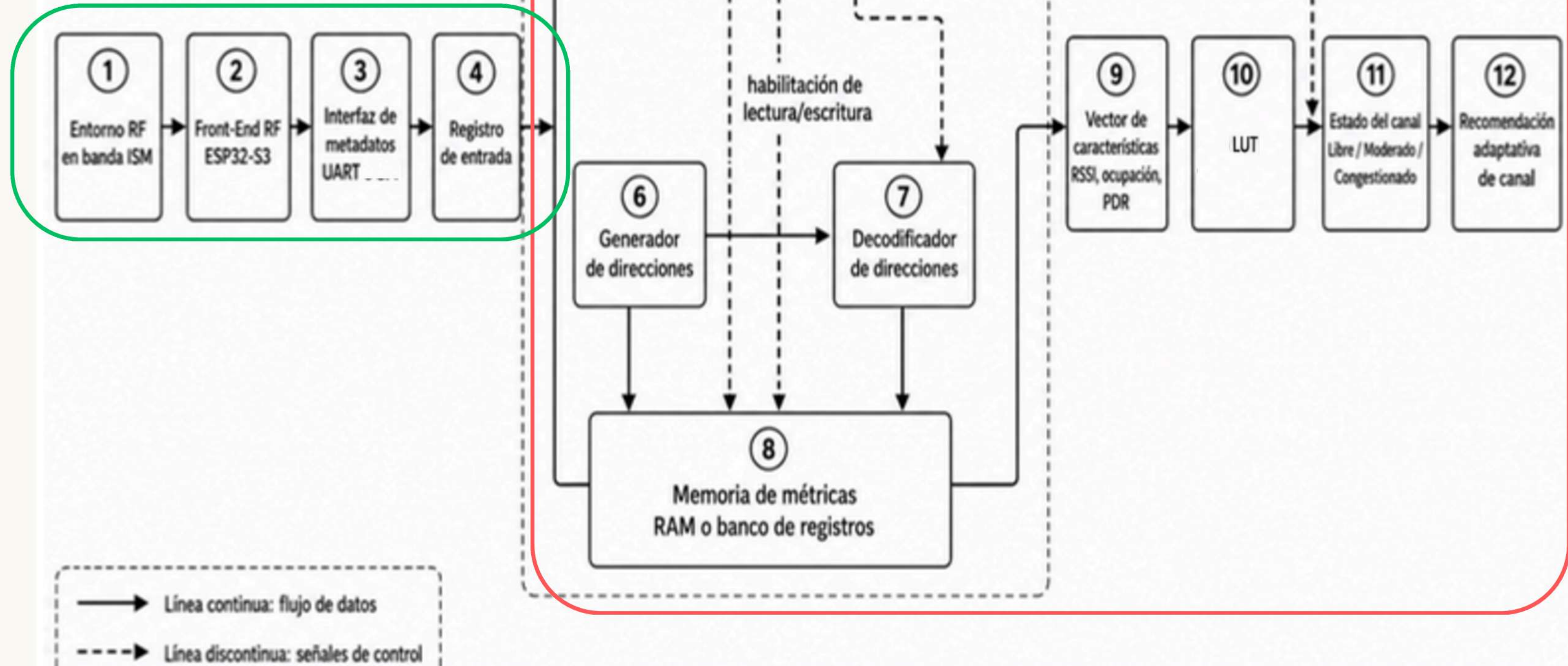
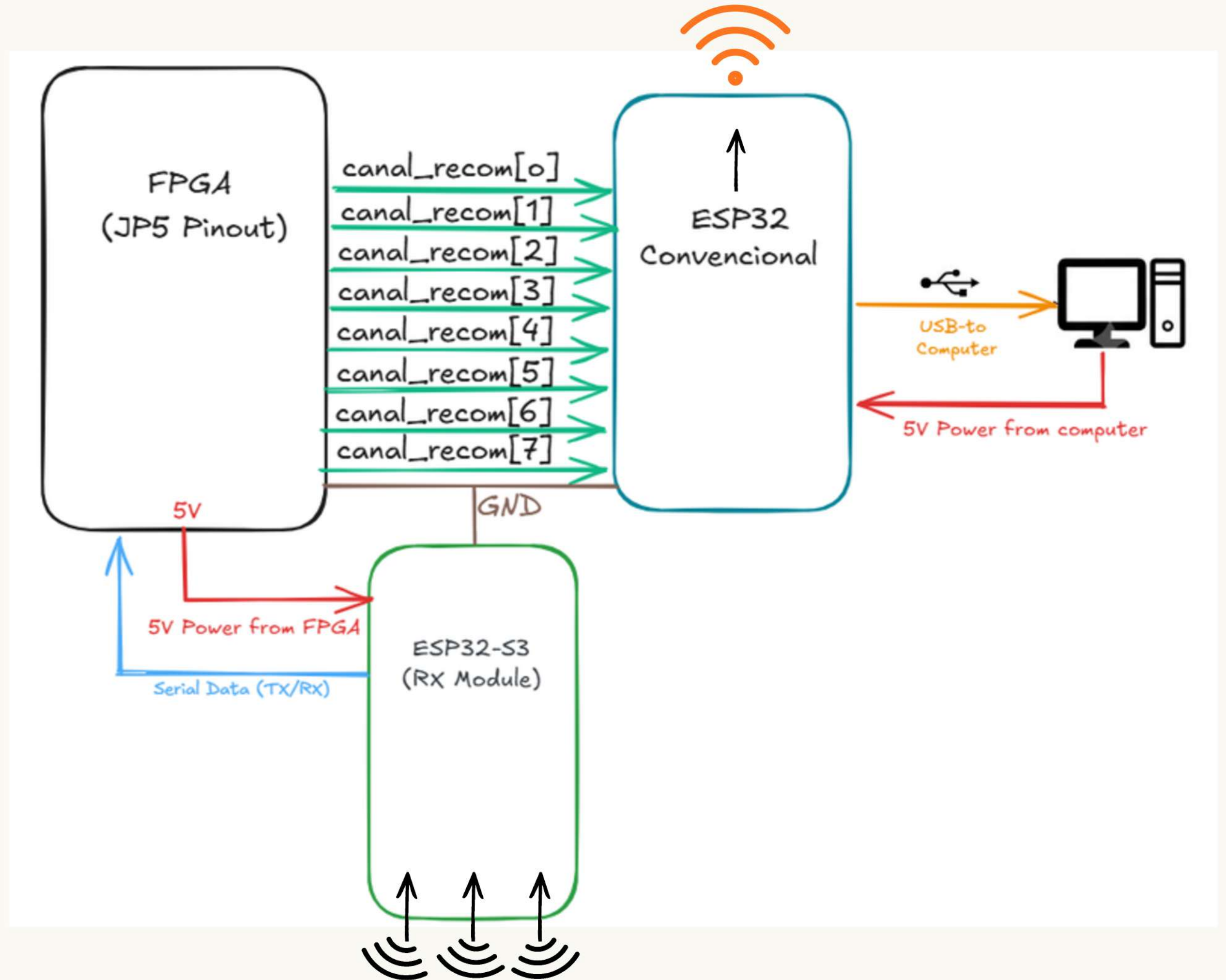
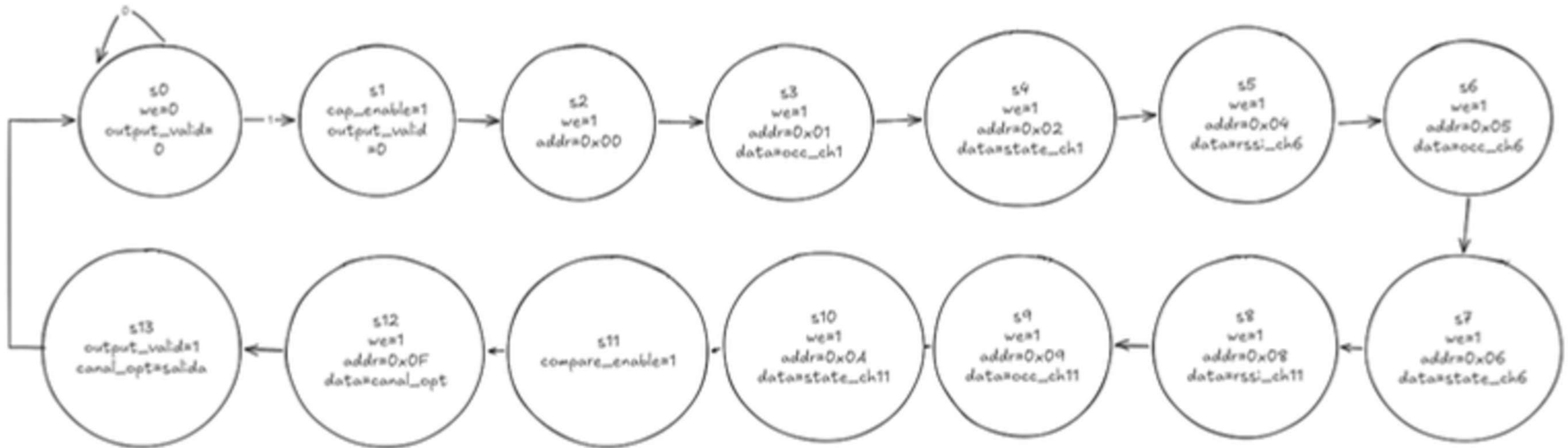


Diagrama de conexiones generales



Máquina de estados finitos (FSM)



s0: IDLE	s7: WRITE_CH6_STATE
s1: CAPTURE	s8: WRITE_CH11_RSSI
s2: WRITE_CH1_RSSI	s9: WRITE_CH11_OCC
s3: WRITE_CH1_OCC	s10: WRITE_CH11_STATE
s4: WRITE_CH1_STATE	s11: COMPARE
s5: WRITE_CH6_RSSI	s12: WRITE_RESULT
s6: WRITE_CH6_OCC	s13: OUTPUT_STATE

(Máquina de Moore)

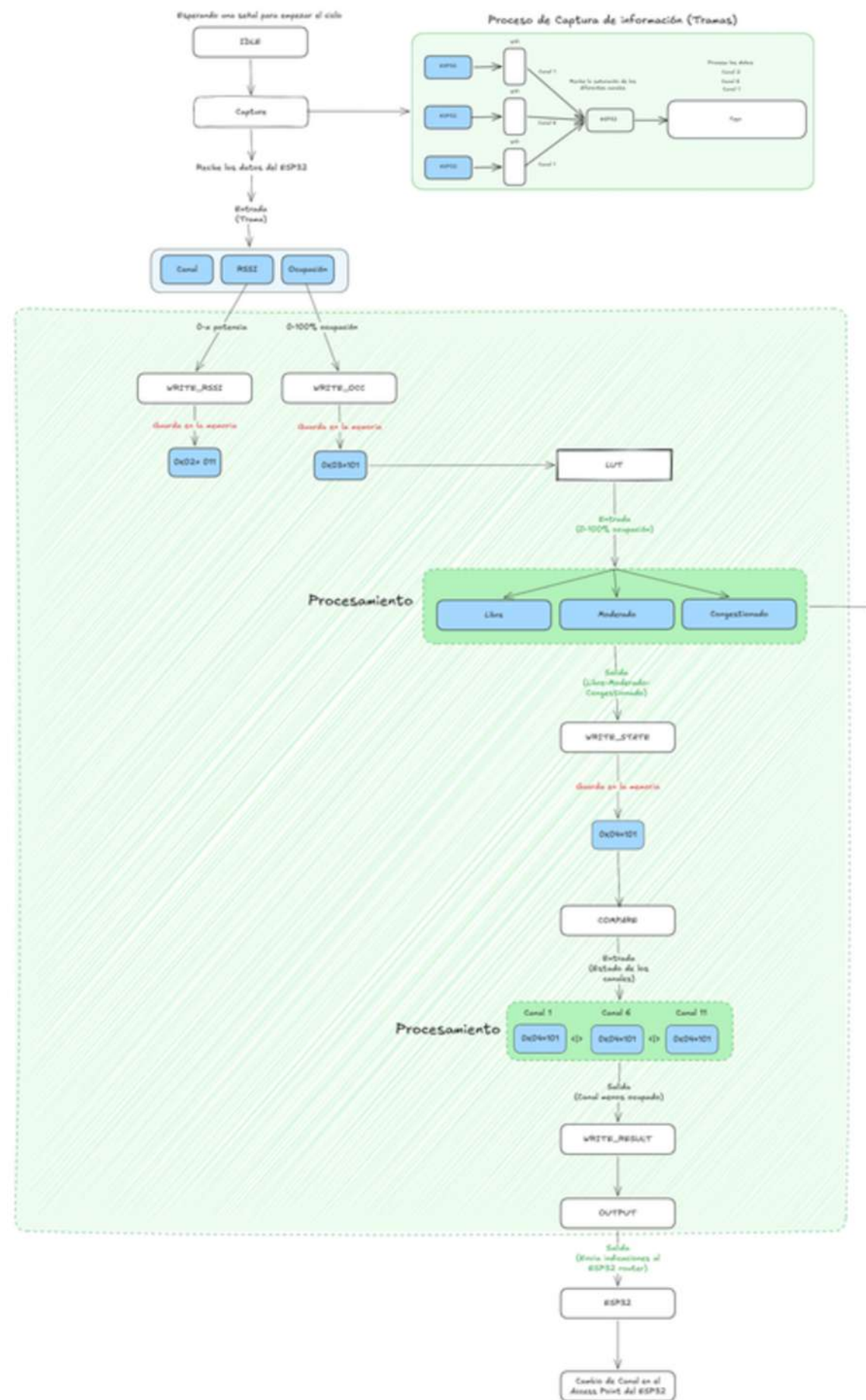
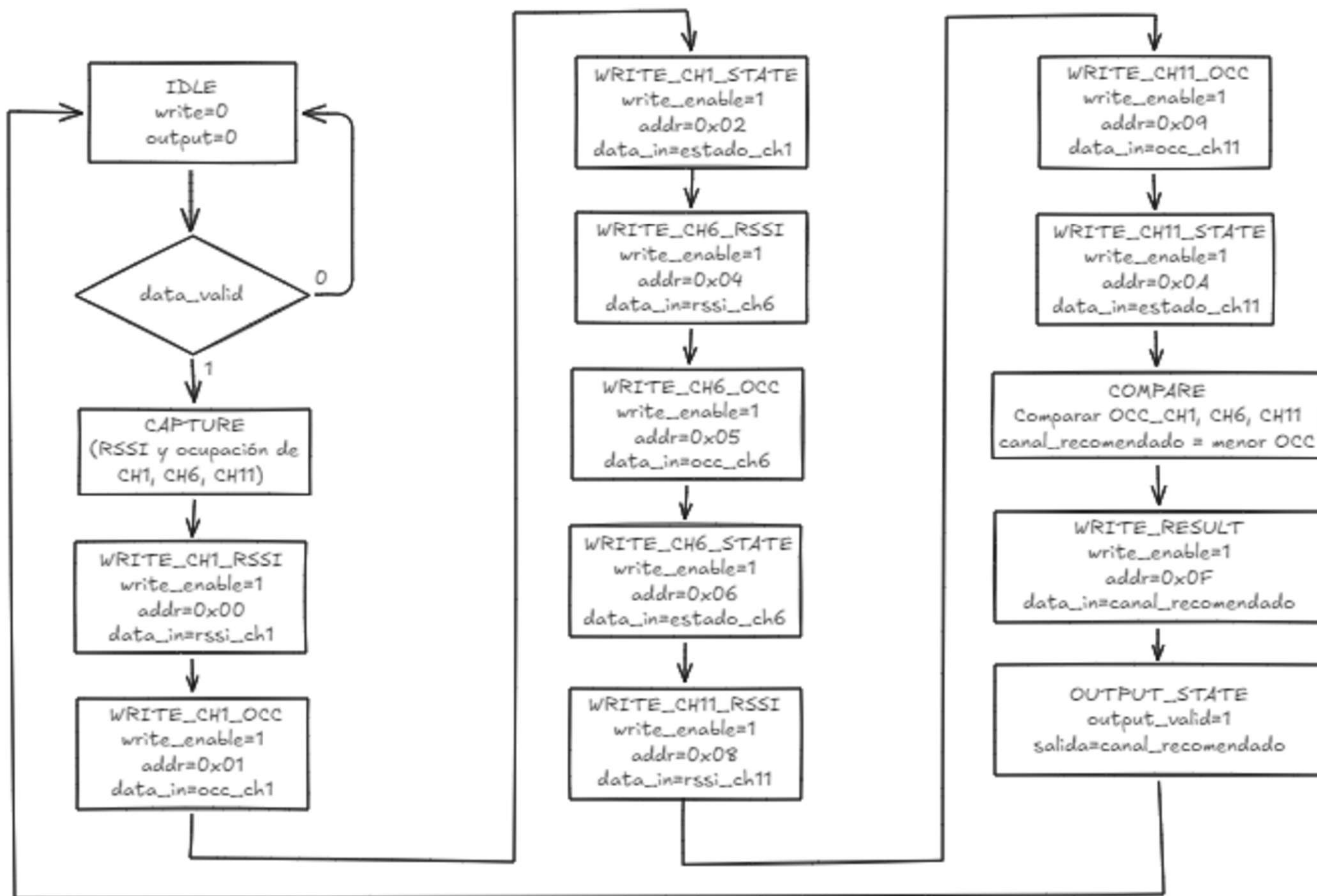


Tabla de estados de la FSM

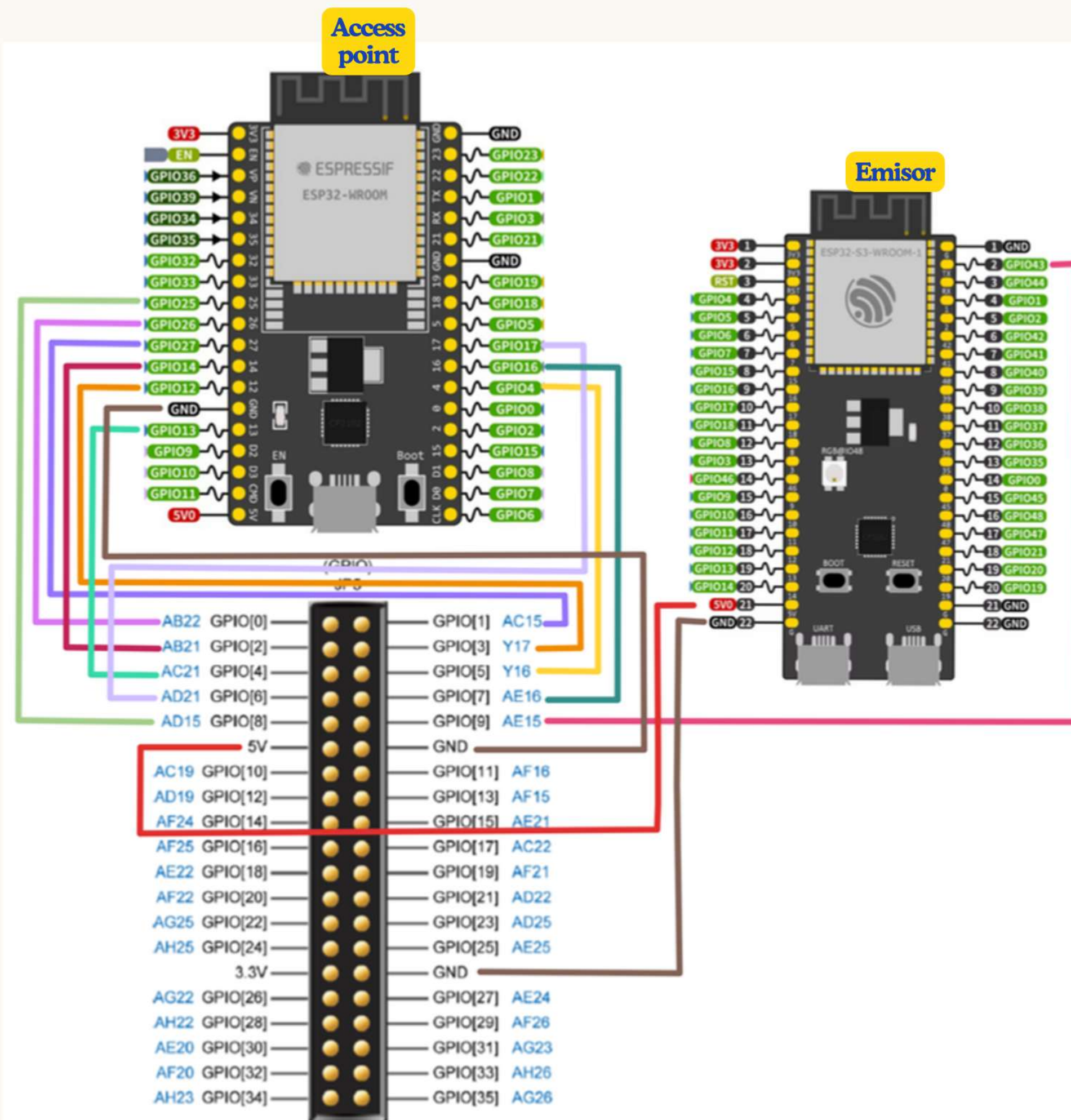
Código	Estado	Función	Siguiente estado
0	IDLE	Espera una trama válida	CAPTURE si data_valid = 1
1	CAPTURE	Captura RSSI y ocupación de los canales 1, 6 y 11	WRITE_CH1_RSSI
2	WRITE_CH1_RSSI	Escribe RSSI del canal 1 en memoria	WRITE_CH1_OCC
3	WRITE_CH1_OCC	Escribe ocupación del canal 1	WRITE_CH1_STATE
4	WRITE_CH1_STATE	Escribe estado clasificado del canal 1	WRITE_CH6_RSSI
5	WRITE_CH6_RSSI	Escribe RSSI del canal 6 en memoria	WRITE_CH6_OCC
6	WRITE_CH6_OCC	Escribe ocupación del canal 6	WRITE_CH6_STATE
7	WRITE_CH6_STATE	Escribe estado clasificado del canal 6	WRITE_CH11_RSSI
8	WRITE_CH11_RSSI	Escribe RSSI del canal 11 en memoria	WRITE_CH11_OCC
9	WRITE_CH11_OCC	Escribe ocupación del canal 11	WRITE_CH11_STATE
10	WRITE_CH11_STATE	Escribe estado clasificado del canal 11	COMPARE
11	COMPARE	Compara ocupaciones y obtiene el canal menos ocupado	WRITE_RESULT
12	WRITE_RESULT	Escribe el canal recomendado en memoria	OUTPUT STATE
13	OUTPUT STATE	Activa output_valid y entrega el resultado	IDLE

Diagrama ASM



Conexiones

Pinout



Signal Name	FPGA Pin No.	Description	I/O Standard
GPIO[0]	PIN_AB22	GPIO Connection DATA[0]	Depending on JP6
GPIO[1]	PIN_AC15	GPIO Connection DATA[1]	Depending on JP6
GPIO[2]	PIN_AB21	GPIO Connection DATA[2]	Depending on JP6
GPIO[3]	PIN_Y17	GPIO Connection DATA[3]	Depending on JP6
GPIO[4]	PIN_AC21	GPIO Connection DATA[4]	Depending on JP6
GPIO[5]	PIN_Y16	GPIO Connection DATA[5]	Depending on JP6
GPIO[6]	PIN_AD21	GPIO Connection DATA[6]	Depending on JP6
GPIO[7]	PIN_AE16	GPIO Connection DATA[7]	Depending on JP6
GPIO[8]	PIN_AD15	GPIO Connection DATA[8]	Depending on JP6
GPIO[9]	PIN_AE15	GPIO Connection DATA[9]	Depending on JP6
GPIO[10]	PIN_AC19	GPIO Connection DATA[10]	Depending on JP6
GPIO[11]	PIN_AF16	GPIO Connection DATA[11]	Depending on JP6
GPIO[12]	PIN_AD19	GPIO Connection DATA[12]	Depending on JP6
GPIO[13]	PIN_AF15	GPIO Connection DATA[13]	Depending on JP6
GPIO[14]	PIN_AF24	GPIO Connection DATA[14]	Depending on JP6
GPIO[15]	PIN_AE21	GPIO Connection DATA[15]	Depending on JP6
GPIO[16]	PIN_AF25	GPIO Connection DATA[16]	Depending on JP6
GPIO[17]	PIN_AC22	GPIO Connection DATA[17]	Depending on JP6
GPIO[18]	PIN_AE22	GPIO Connection DATA[18]	Depending on JP6
GPIO[19]	PIN_AF21	GPIO Connection DATA[19]	Depending on JP6
GPIO[20]	PIN_AF22	GPIO Connection DATA[20]	Depending on JP6
GPIO[21]	PIN_AD22	GPIO Connection DATA[21]	Depending on JP6
GPIO[22]	PIN_AG25	GPIO Connection DATA[22]	Depending on JP6
GPIO[23]	PIN_AD25	GPIO Connection DATA[23]	Depending on JP6
GPIO[24]	PIN_AH25	GPIO Connection DATA[24]	Depending on JP6
GPIO[25]	PIN_AE25	GPIO Connection DATA[25]	Depending on JP6
GPIO[26]	PIN_AG22	GPIO Connection DATA[26]	Depending on JP6
GPIO[27]	PIN_AE24	GPIO Connection DATA[27]	Depending on JP6
GPIO[28]	PIN_AH22	GPIO Connection DATA[28]	Depending on JP6
GPIO[29]	PIN_AF26	GPIO Connection DATA[29]	Depending on JP6
GPIO[30]	PIN_AE20	GPIO Connection DATA[30]	Depending on JP6
GPIO[31]	PIN_AG23	GPIO Connection DATA[31]	Depending on JP6
GPIO[32]	PIN_AF20	GPIO Connection DATA[32]	Depending on JP6
GPIO[33]	PIN_AH26	GPIO Connection DATA[33]	Depending on JP6
GPIO[34]	PIN_AH23	GPIO Connection DATA[34]	Depending on JP6
GPIO[35]	PIN_AG26	GPIO Connection DATA[35]	Depending on JP6

Pinout

SEÑAL FPGA	PIN FPGA	GPIO ESP32	FUNCIÓN
canal_recomendado[0]	AB22 (GPIO0)	GPIO26	Bit 0 del canal recomendado
canal_recomendado[1]	AC15 (GPIO1)	GPIO27	Bit 1 del canal recomendado
canal_recomendado[2]	AB21 (GPIO2)	GPIO14	Bit 2 del canal recomendado
canal_recomendado[3]	Y17 (GPIO3)	GPIO12	Bit 3 del canal recomendado
canal_recomendado[4]	AC21 (GPIO4)	GPIO13	Bit 4 del canal recomendado
canal_recomendado[5]	Y16 (GPIO5)	GPIO4	Bit 5 del canal recomendado
canal_recomendado[6]	AD21 (GPIO6)	GPIO17	Bit 6 del canal recomendado
canal_recomendado[7]	AE16 (GPIO7)	GPIO16	Bit 7 del canal recomendado
output_valid	AD15 (GPIO8)	GPIO25	Validación de datos

Conexión con ESP32-S3 (emisor)			
UART RX (desde ESP32-S3)	AE15 (GPIO1)	GPIO43 (TX del ESP32-S3)	Recepción de datos de ocupación y RSSI
Alimentación y tierra			
Tierra común	GND	GND	Referencia eléctrica común
Alimentación	5V FPGA	5V ESP32-S3	Alimentación del módulo emisor

Simulaciones Y métrica

Simulación

The screenshot displays the Quartus Prime Lite Edition interface with the 'Compilation Report - ISM_ANALYZER' open. The report shows the following data:

Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time (on all processors)
1. Analysis_inthesis	00:00:09	1.0	4857 MB	00:00:21
2. Filter	00:00:21	1.0	6719 MB	00:00:36
3. Assembler	00:00:03	1.0	4702 MB	00:00:53
4. Timing Analyzer	00:00:02	1.1	5007 MB	00:00:02
5. EDA Netlist Writer	00:00:00	1.0	4661 MB	00:00:00
6. Total	00:00:35	--	--	00:01:02

The 'Messages' window at the bottom shows the following log:

```
Running Quartus Prime Analysis & Synthesis
Command: quartus_map --read_settings_files-on --write_settings_files-off ISM_ANALYZER -c ISM_ANALYZER
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
20030 Parallel compilation is enabled and will use 16 of the 16 processors detected
12021 Found 1 design units, including 1 entities, in source file address_decoder.v
12021 Found 1 design units, including 1 entities, in source file decision_lut.v
12021 Found 1 design units, including 1 entities, in source file metric_memory.v
12021 Found 1 design units, including 1 entities, in source file channel_comparator.v
12021 Found 1 design units, including 1 entities, in source file ism_analyzer_top.v
12127 Elaborating entity "ism_analyzer_top" for the top level hierarchy
12128 Elaborating entity "decision_lut" for hierarchy "decision_lut::LUT_Cnx1"
12128 Elaborating entity "channel_comparator" for hierarchy "channel_comparator::CMP"
12128 Elaborating entity "metric_memory" for hierarchy "metric_memory::MPX"
10240 Verilog HDL Always Construct warning at metric_memory.v(15): Inferring latch(es) for variable "t1", which holds its previous value in one or more paths through the always construct
```

The screenshot displays the Quartus Prime Lite Edition interface with the 'Compilation Report - ISM_ANALYZER' open. The report shows the following data:

Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time (on all processors)
1. Analysis_inthesis	00:00:09	1.0	4857 MB	00:00:21
2. Filter	00:00:21	1.0	6719 MB	00:00:36
3. Assembler	00:00:03	1.0	4702 MB	00:00:53
4. Timing Analyzer	00:00:02	1.1	5007 MB	00:00:02
5. EDA Netlist Writer	00:00:00	1.0	4661 MB	00:00:00
6. Total	00:00:35	--	--	00:01:02

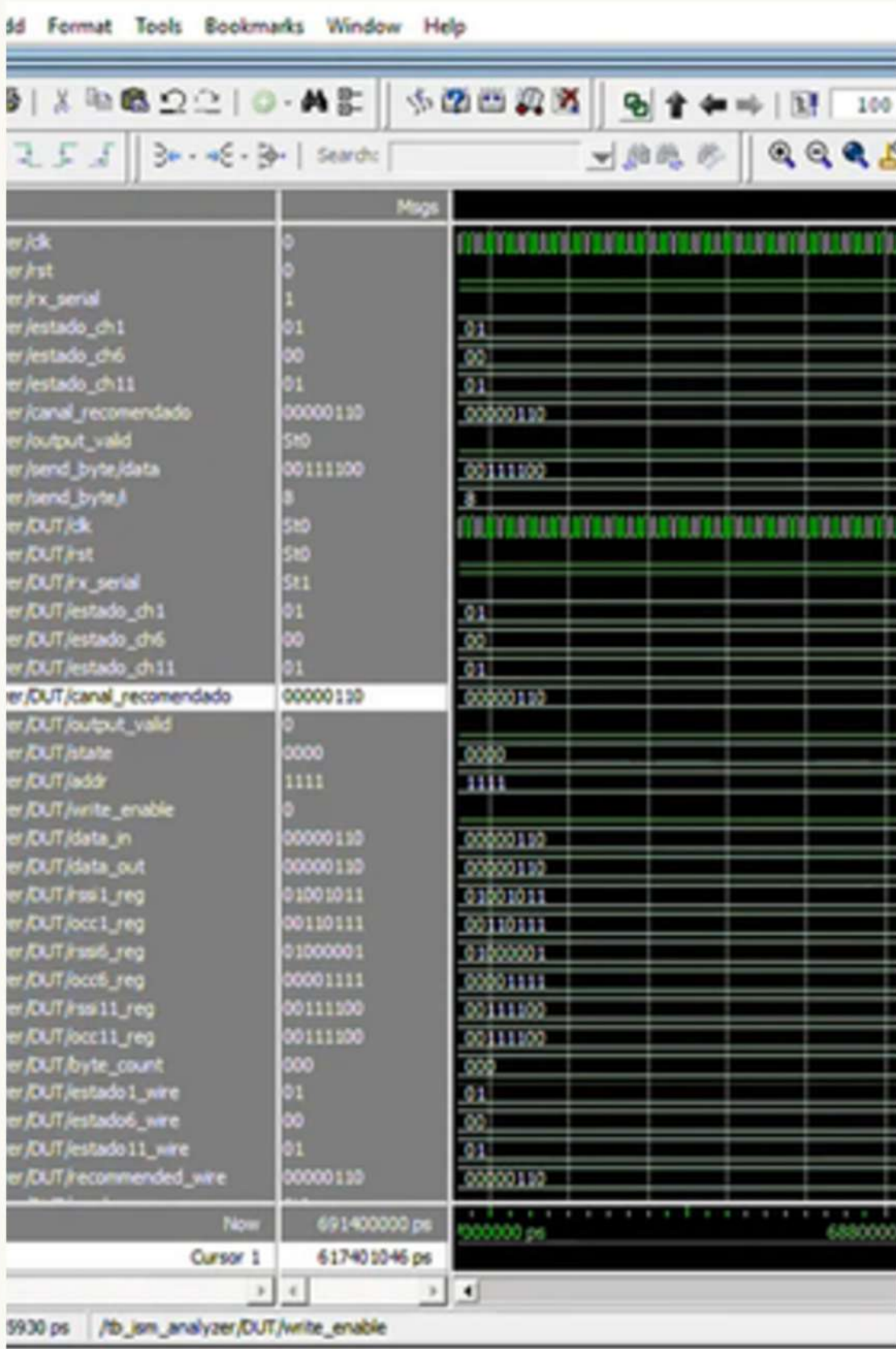
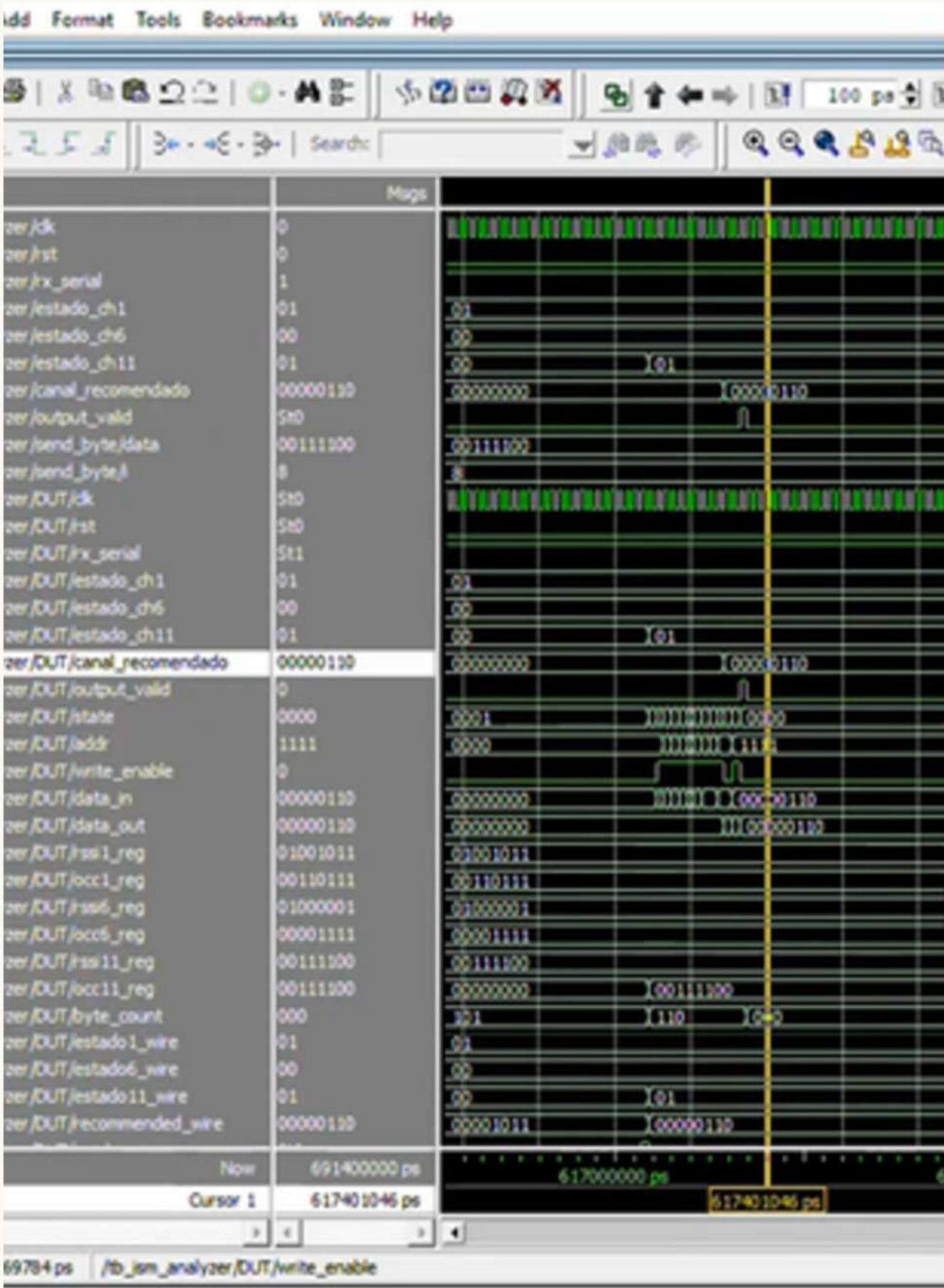
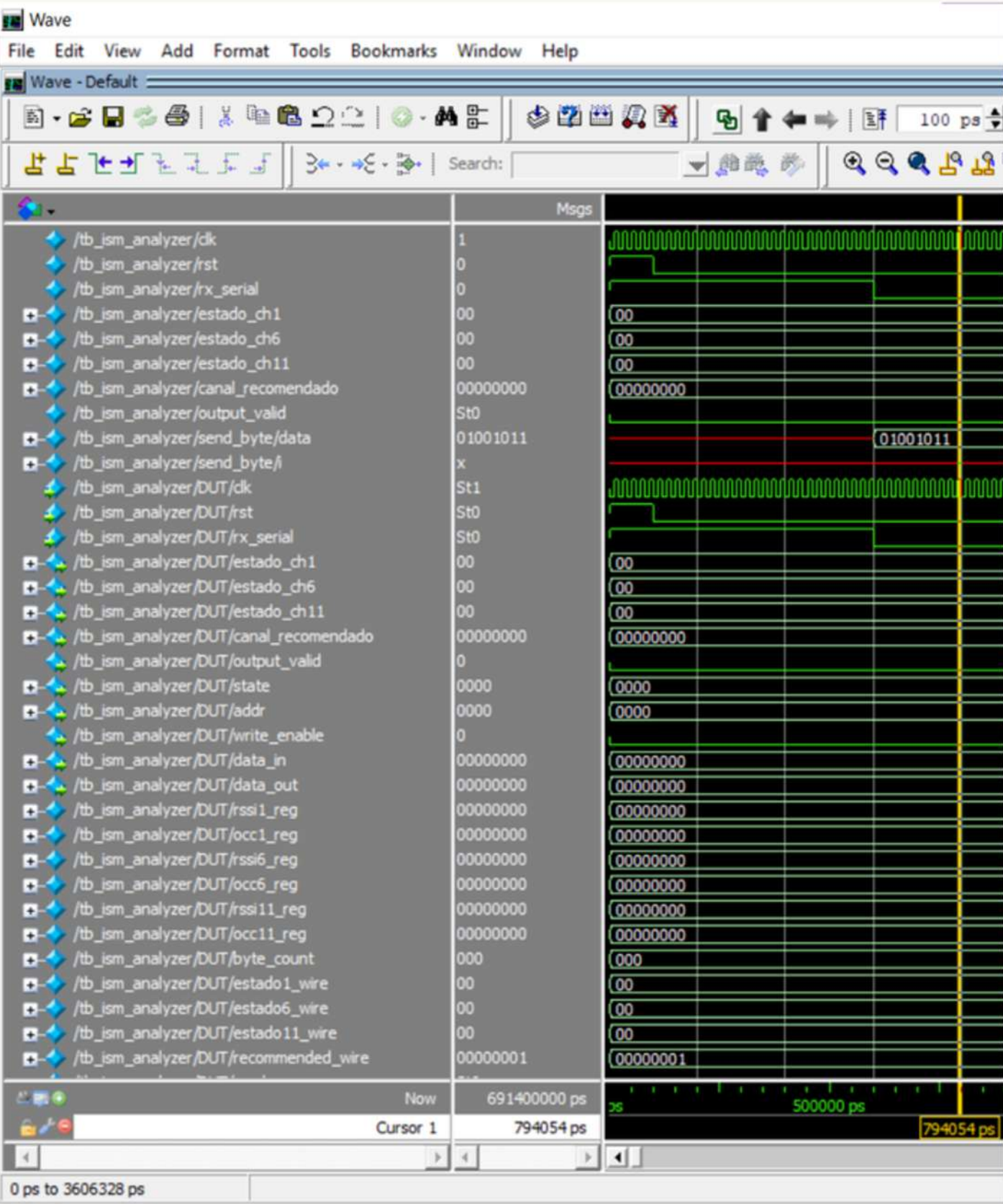
The 'Messages' window at the bottom shows the following log:

```
Running Quartus Prime Analysis & Synthesis
Command: quartus_map --read_settings_files-on --write_settings_files-off ISM_ANALYZER -c ISM_ANALYZER
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
20030 Parallel compilation is enabled and will use 16 of the 16 processors detected
12021 Found 1 design units, including 1 entities, in source file address_decoder.v
12021 Found 1 design units, including 1 entities, in source file decision_lut.v
12021 Found 1 design units, including 1 entities, in source file metric_memory.v
12021 Found 1 design units, including 1 entities, in source file channel_comparator.v
12021 Found 1 design units, including 1 entities, in source file ism_analyzer_top.v
12127 Elaborating entity "ism_analyzer_top" for the top level hierarchy
12128 Elaborating entity "decision_lut" for hierarchy "decision_lut::LUT_Cnx1"
12128 Elaborating entity "channel_comparator" for hierarchy "channel_comparator::CMP"
12128 Elaborating entity "metric_memory" for hierarchy "metric_memory::MPX"
10240 Verilog HDL Always Construct warning at metric_memory.v(15): Inferring latch(es) for variable "t1", which holds its previous value in one or more paths through the always construct
```

A system monitor window is overlaid on the right side of the screen, showing the following performance metrics:

- CPU: 17% 3.69 GHz
- Memoria: 13.3/15.4 GB (86%)
- Disco 0 (G: C): 150 (MB/s)
- Wi-Fi: 6.37 Kbps
- GPU 0: AMD Radeon (TM) 7 5825U with Radeon Graphics
- Velocidad de base: 2.00 GHz
- Procesos: 322
- Subprocesos: 5332
- Identificadores: 170625
- Procesadores lógicos: 16
- Virtualización: Habilitado
- Caché L1: 512 KB
- Caché L2: 4.0 MB
- Caché L3: 16.0 MB

Resultado de la simulación tras aplicación del testbench



Resultado de la lógica de decisión del prototipo físico en monitor serial.

The image displays two side-by-side screenshots of the Arduino IDE interface, showing code for an ESP32-based system and its serial monitor output.

Left Window (ESP32S3 Dev Module): The code in `sketch_jun10c.ino` defines constants for channel occupancy and a 3-second interval. The `setup()` function initializes `Serial2` on pins 43 (TX) and 42 (RX). The `loop()` function sends a sequence of six bytes (RSSI_CH1, OCC_CH1, RSSI_CH6, OCC_CH6, RSSI_CH11, OCC_CH11) via `Serial2`, flushes the buffer, and delays for 3 seconds.

Right Window (ESP32 Dev Module): The code in `sketch_jun10a.ino` defines a global `datoListo` variable. The `setup()` function initializes `Serial` at 115200 baud, configures `PIN_VALID` as an input, and attaches an interrupt. It then prints a message and waits for a signal. The `loop()` function is partially visible.

Serial Monitor Output (Right Window): The output shows the message "[FPGA ALERTA]: Lectura de Pines en Paralelo" followed by a list of GPIO pin states:

- Bit 7 (GPIO 16): 0
- Bit 6 (GPIO 17): 0
- Bit 5 (GPIO 4): 0
- Bit 4 (GPIO 13): 0
- Bit 3 (GPIO 12): 0
- Bit 2 (GPIO 14): 1
- Bit 1 (GPIO 27): 1
- Bit 0 (GPIO 26): 0

The final output line is "--> Resultado Combinado: CH6".

Testbench aplicado en la simulación

```
D:/ProyectoPrueba/tb_ism_analyzer.v - Default
Ln#
1  `timescale 1ns/1ps
2
3  module tb_ism_analyzer;
4
5      // Señales de prueba
6      reg clk;
7      reg rst;
8      reg rx_serial;
9
10     // Cables para observar las salidas
11     wire [1:0] estado_ch1, estado_ch6, estado_ch11;
12     wire [7:0] canal_recomendado;
13     wire output_valid;
14
15     // 1. Instanciamos tu diseño principal (Device Under Test)
16     ism_analyzer_top DUT (
17         .clk(clk),
18         .rst(rst),
19         .rx_serial(rx_serial),
20         .estado_ch1(estado_ch1),
21         .estado_ch6(estado_ch6),
22         .estado_ch11(estado_ch11),
23         .canal_recomendado(canal_recomendado),
24         .output_valid(output_valid)
25     );
26
27     // 2. Generador de Reloj (50 MHz = periodo de 20ns)
28     always #10 clk = ~clk;
29
30     // 3. Tarea para simular el envío UART desde un ESP32 a 115200 baudios
31     // 1 bit a 115200 baudios dura aprox 8680 nanosegundos
32     task send_byte;
33         input [7:0] data;
34         integer i;
35         begin
```

```
D:/ProyectoPrueba/tb_ism_analyzer.v - Default
Ln#
35         begin
36             rx_serial = 0; // Start bit
37             #8680;
38             for(i=0; i<8; i=i+1) begin
39                 rx_serial = data[i]; // Data bits
40                 #8680;
41             end
42             rx_serial = 1; // Stop bit
43             #8680;
44             #20000; // Pequeña pausa entre bytes
45         end
46     endtask
47
48     // 4. Secuencia de la simulación
49     initial begin
50         // Inicialización
51         clk = 0;
52         rst = 1;
53         rx_serial = 1; // El estado de reposo del UART es 1
54
55         #100; // Esperamos 100ns
56         rst = 0; // Apagamos el reset
57         #500;
58
59         $display("Iniciando inyección de métricas UART...");
60
61         // Caso de prueba del PDF:
62         // CH1: RSSI 75, Ocupación 55
63         send_byte(8'd75);
64         send_byte(8'd55);
65
66         // CH6: RSSI 65, Ocupación 15
67         send_byte(8'd65);
68         send_byte(8'd15);
69
70         // CH11: RSSI 60, Ocupación 60
71         send_byte(8'd60);
72         send_byte(8'd60);
73
74         // Esperamos a que la FSM procese todo
75         #50000;
76
77         $display("Simulación terminada.");
78         $stop; // Detenemos la gráfica
79     end
80
81 endmodule
```

Canal 1: 55% de ocupación
Canal 6: 15% de ocupación
Canal 11: 60% de ocupación

Métricas de Evaluación y Procedimiento de Medición

MÉTRICAS DE EVALUACIÓN

El desempeño del sistema fue evaluado mediante tres métricas principales.

MÉTRICA	SÍMBOLO / FÓRMULA	DESCRIPCIÓN	UNIDAD	OBJETIVO
Porcentaje de Ocupación de Canal (CO)	$CO(\%) = \frac{T_{ocupado}}{T_{total}} \times 100$	Porcentaje de tiempo en el que el canal está ocupado por alguna transmisión.	%	Identificar el canal menos ocupado.
Tasa de Entrega de Paquetes (PDR)	$PDR(\%) = \frac{N_{recibidos}}{N_{enviados}} \times 100$	Relación entre paquetes recibidos correctamente y enviados.	%	Evaluar la confiabilidad del enlace.
Latencia de Decisión (T_d)	$T_d = t_{salida} - t_{entrada}$	Tiempo que tarda el sistema en decidir el canal óptimo desde que inicia el análisis.	μs	Medir la rapidez de respuesta del sistema.



ESCENARIOS DE EVALUACIÓN

- Libre:** entorno sin interferencia intencional.
- Moderado:** tráfico generado por hotspot o streaming.
- Congestionado:** interferencia activa mediante jammer (ESP32 Jammer v1.0).

CANALES IEEE 802.11 ANALIZADOS

Canal	Frecuencia Central	Ancho de Banda
CH1	2.412 GHz	20 MHz
CH6	2.437 GHz	
CH11	2.462 GHz	



Bandas ISM de 2.4 GHz



OBJETIVO GENERAL

Validar la capacidad del sistema para detectar ocupación de canal, seleccionar el canal óptimo y mejorar el desempeño del enlace en presencia de interferencia.

PROCEDIMIENTO DE MEDICIÓN

1 Configuración del sistema

- Se conecta el AP (ESP32) y la FPGA (DE2-115).
- La FPGA inicia el análisis de los canales CH1, CH6 y CH11.



2 Generación de interferencia

- Se activa el ESP32 Jammer v1.0.
- El jammer transmite en el canal objetivo (ej. CH6) para generar congestión.



3 Medición del porcentaje de ocupación

- La FPGA mide el tiempo en estado "ocupado" y "libre" del canal.
- Datos enviados por UART al osciloscopio.



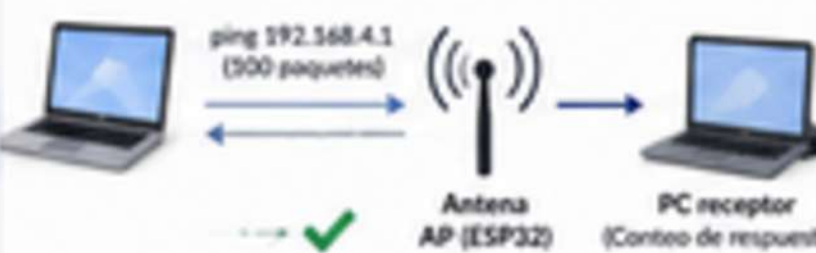
4 Medición de latencia de decisión

- El osciloscopio registra las marcas de tiempo:
 - $t_{entrada}$: inicio del análisis.
 - t_{salida} : canal recomendado.
- $T_d = t_{salida} - t_{entrada}$



5 Medición de PDR

- Se envían 100 paquetes ICMP (ping).
- Se registra el número de paquetes recibidos.
- Se calcula PDR(%).



EQUIPOS UTILIZADOS

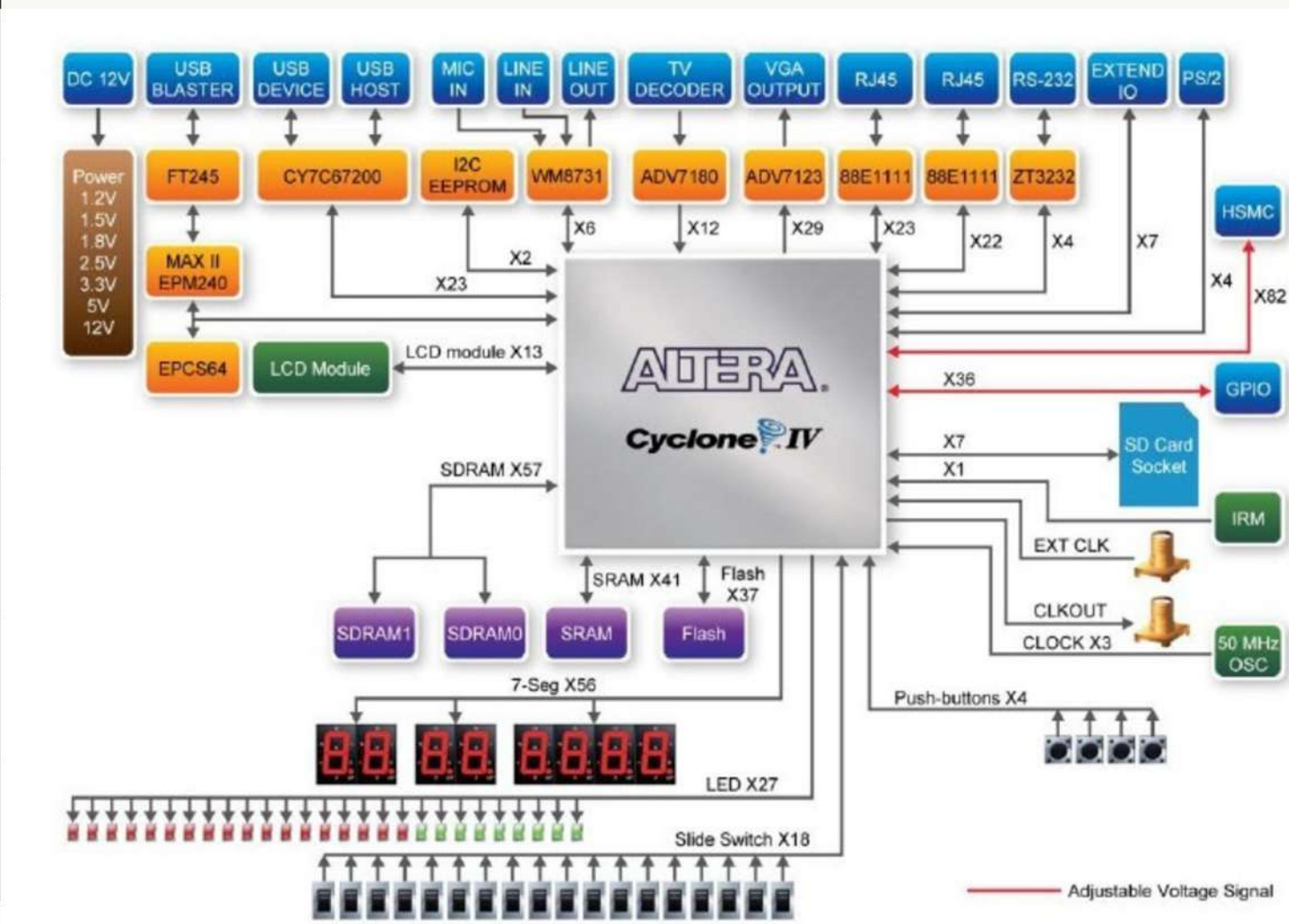
• ESP32 AP • ESP32 Jammer v1.0 • FPGA DE2-115 • Osciloscopio DSOX3024A • PC (Python / MATLAB)

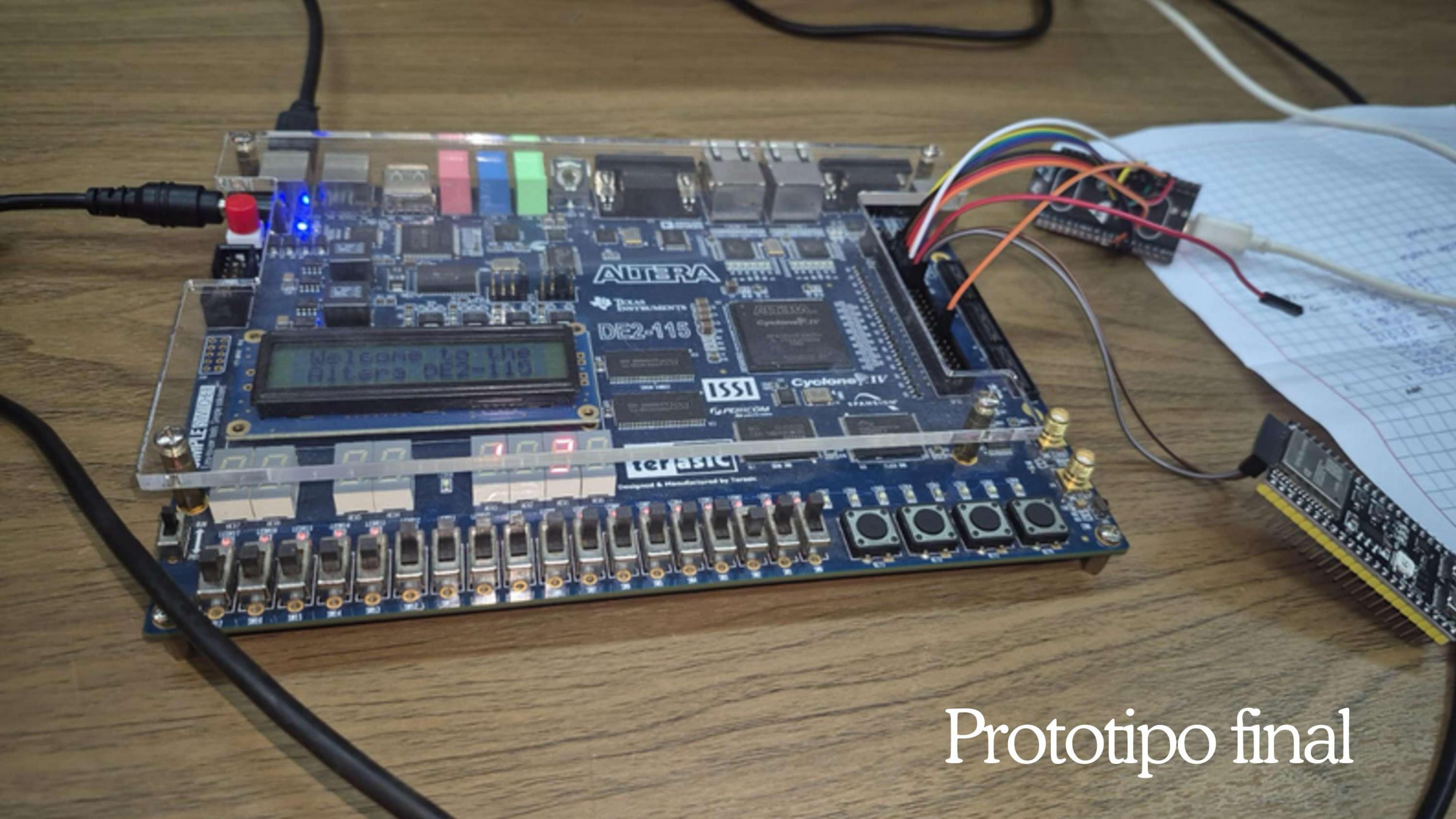
Implementación

RECURSOS OCUPADOS DEL FPGA

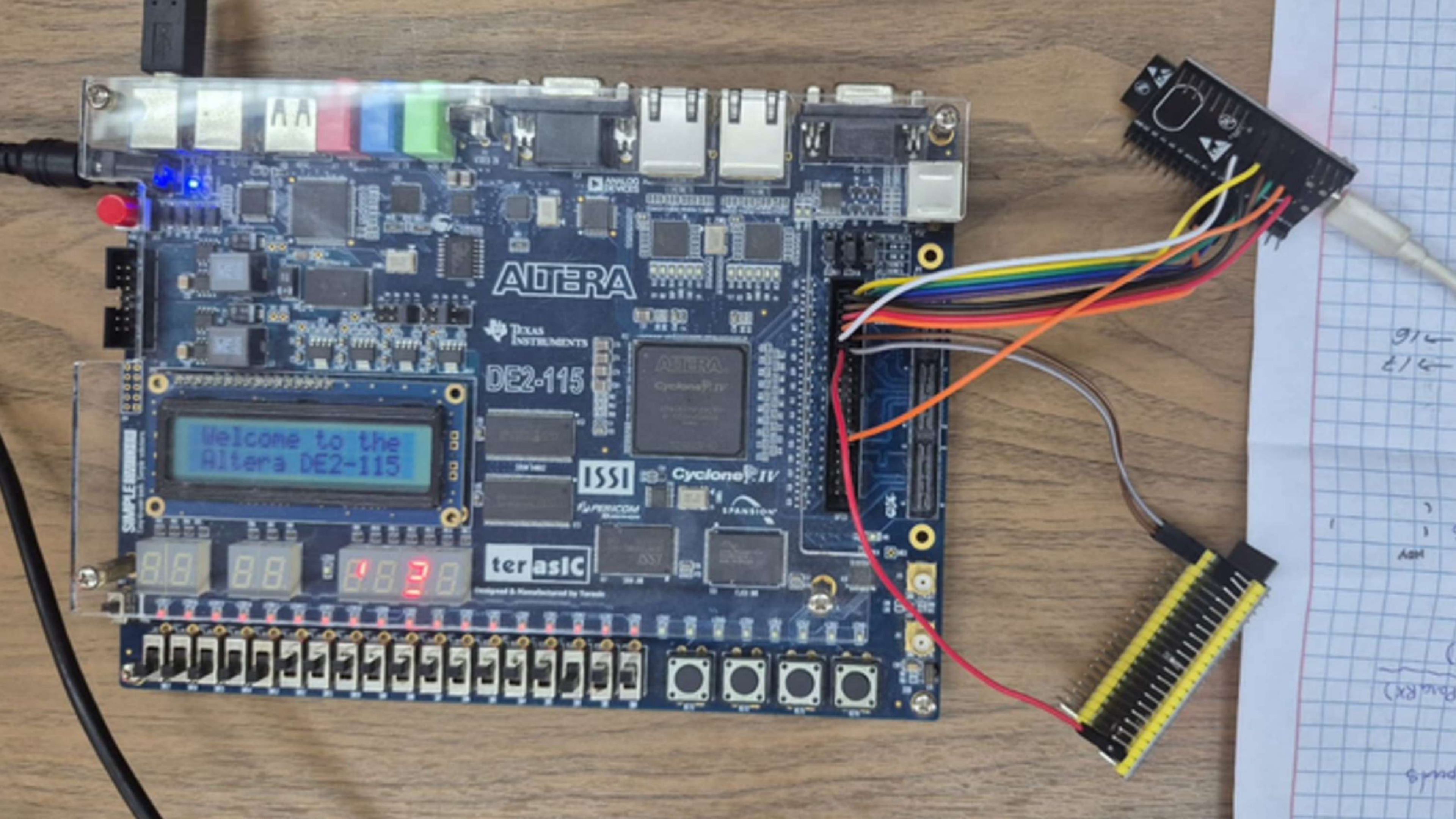
El reporte de compilación muestra una ocupación muy reducida de los recursos disponibles de la FPGA. Se emplearon únicamente 350 elementos lógicos, 235 registros y 43 pines de entrada/salida, manteniendo un amplio margen de capacidad libre para futuras mejoras del sistema.

RECURSO	UTILIZADO	DISPONIBLE	PORCENTAJE
Elementos lógicos (LUTs)	350	114 480	< 1 %
Funciones combinacionales	272	114 480	< 1 %
Registros dedicados (Flip-Flops)	235	114 480	< 1 %
Pines de entrada/salida (I/O)	43	529	8%
Bits de memoria RAM	0	3 981 312	0%
Bloques de memoria M9K	0	432	0%
Multiplicadores embebidos de 9 bits	0	532	0%
PLLs	0	4	0%





Prototipo final



Resultados



Entregable 3: Analizador de ocupación de bandas ISM y selección adaptativa de ca

SaiSai



físico de reset nos

Watch on  YouTube

Justificación

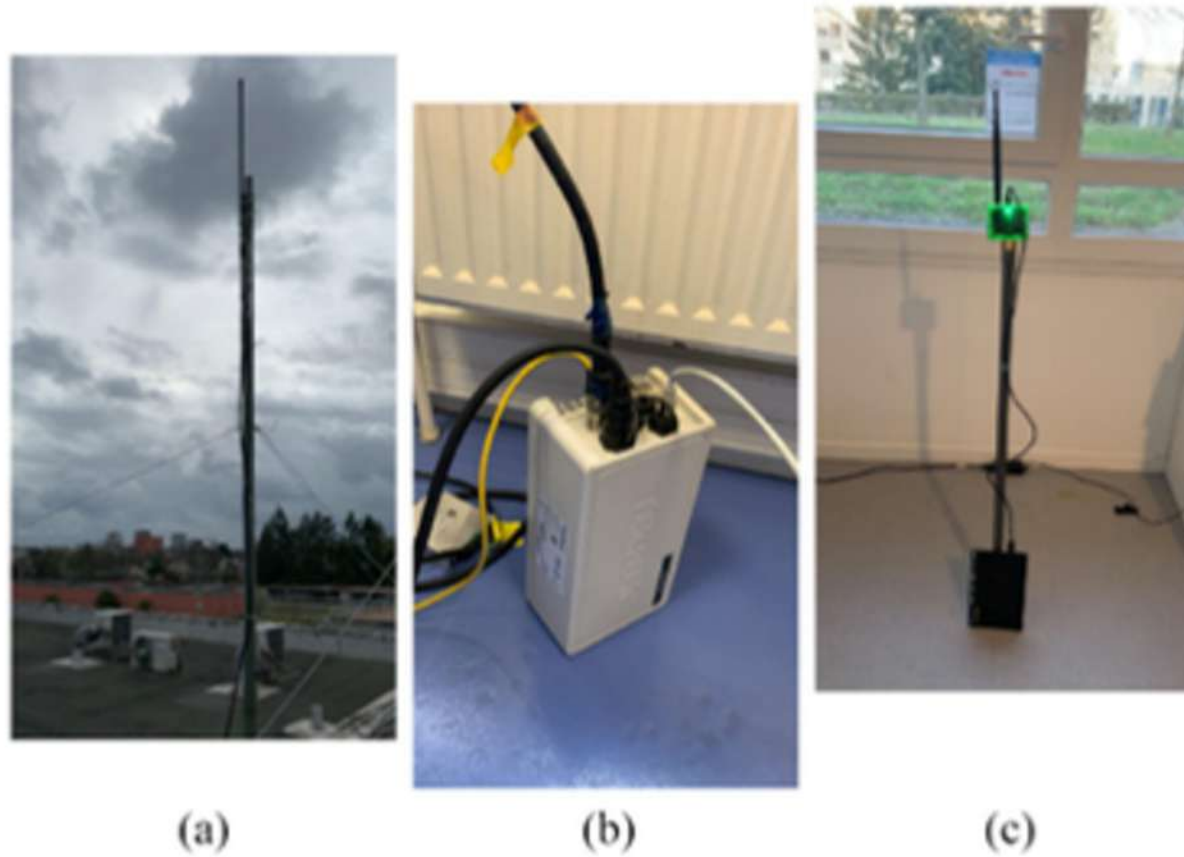


Fig. 3. Views from the end node and gateway sites. (a) Position of the gateway monopole antenna fixed on the roof of the IETR lab. (b) Gateway connected to the antenna cable. (c) Packaged Pycom device is fixed on a rod and connected to a battery inside a building.

First, the network is joined with an over-the-air-activation (OTA-A). Then, the proposed algorithms are executed sequen-

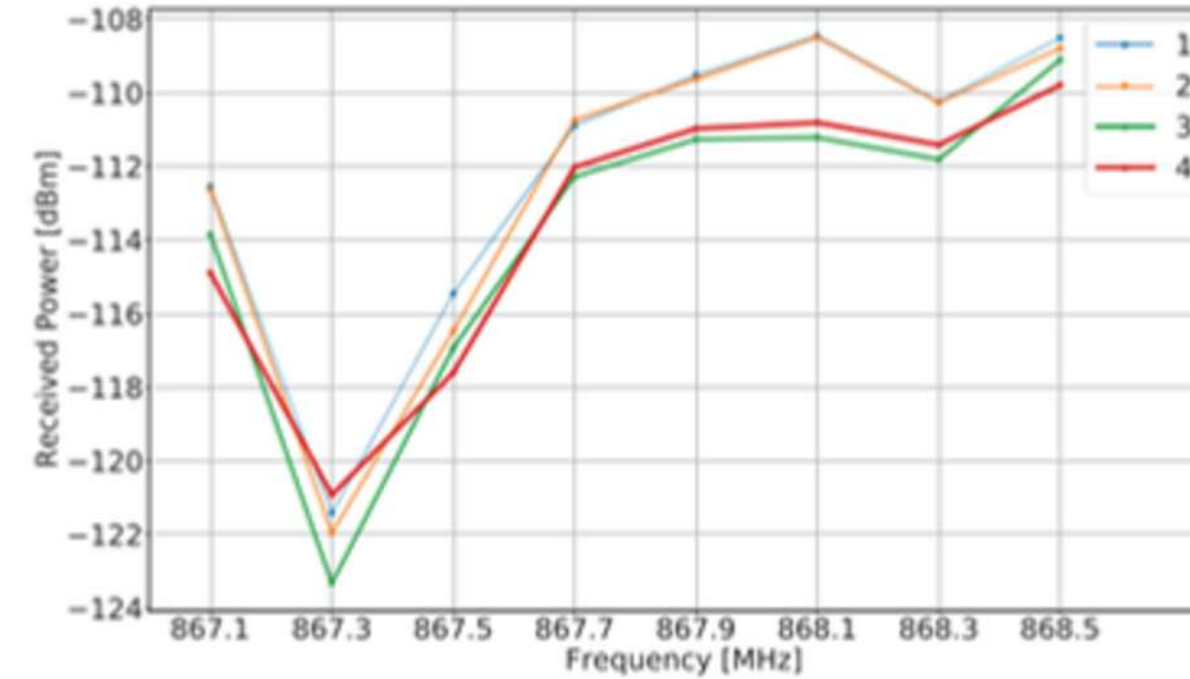


Fig. 4. Stability of the CSI shape throughout the 800 steps (one CSI every 200 steps) in the stationary scenario.

TABLE V
NUMBER OF SUCCESSFULLY TRANSMITTED AND LOST
PACKETS IN THE STATIONARY SCENARIO

Algorithm	Succeed	Lost	Success rate
Uniform	668	132	83.5%
UCB	761	39	95.1%
QoC-A	768	32	96.0%

Justificación normativa

Conclusiones de la Demostración Funcional

La implementación del sistema en hardware real permitió validar el funcionamiento del **analizador inteligente de ocupación en bandas ISM** y su capacidad de **selección adaptativa de canal**.

-  **Selección adaptativa efectiva**
El sistema identifica el canal con menor ocupación y lo recomienda correctamente, mejorando el desempeño del enlace.
-  **Baja latencia de decisión**
La arquitectura digital implementada en FPGA proporciona tiempos de respuesta del orden de microsegundos.
-  **Uso eficiente de recursos**
El diseño requiere un bajo porcentaje de los recursos disponibles de la FPGA, permitiendo futuras ampliaciones.
-  **Funcionamiento robusto y estable**
Las pruebas en distintos escenarios confirman la estabilidad y confiabilidad del sistema en condiciones reales.



Recursos Ocupados en los ESP32

El sistema utiliza dos módulos ESP32 con funciones complementarias. El ESP32-S3 realiza el escaneo de redes WiFi y envía las métricas a la FPGA mediante UART, mientras que el ESP32 receptor interpreta el canal recomendado y configura un punto de acceso inalámbrico utilizando dicho canal.

PROGRAMA	FUNCIÓN PRINCIPAL	PLATAFORMA
scanner_sender.ino	Escaneo de redes WiFi y envío de métricas a la FPGA mediante UART	ESP32-S3 DevKitC-1
ap_receiver.ino	Recepción del canal recomendado desde la FPGA y configuración de un punto de acceso WiFi	ESP32 Dev Module
simulator_sender.ino	Envío de escenarios simulados a la FPGA para pruebas y validación	ESP32

Memoria utilizada

Los firmwares compilados presentan tamaños inferiores a 1 MB, lo que representa un uso moderado de la memoria Flash disponible en los ESP32.

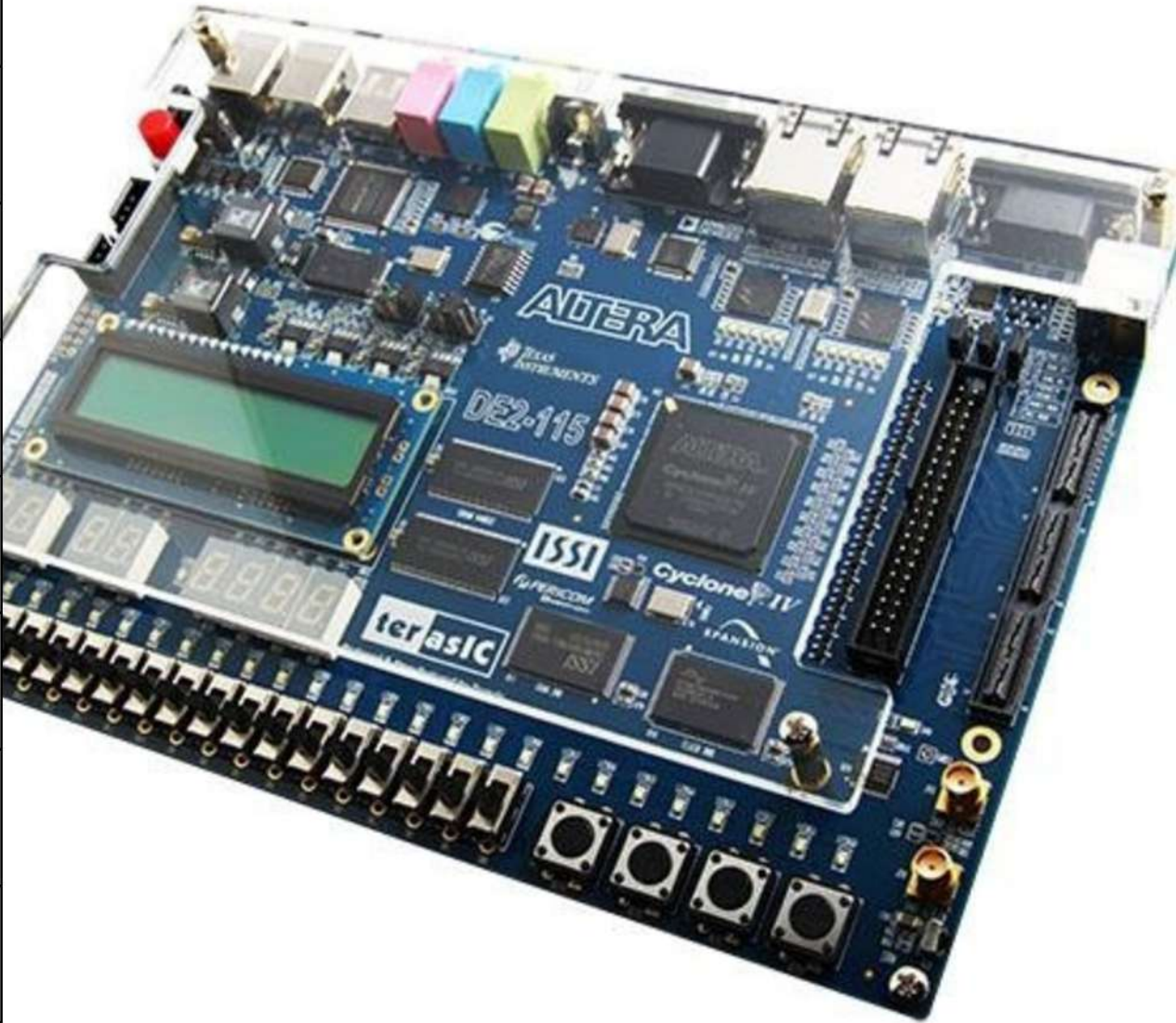
PROGRAMA	TAMAÑO DEL FIRMWARE (.bin)
scanner_sender	676 528 bytes (660.67 KB)
ap_receiver	733 920 bytes (716.72 KB)



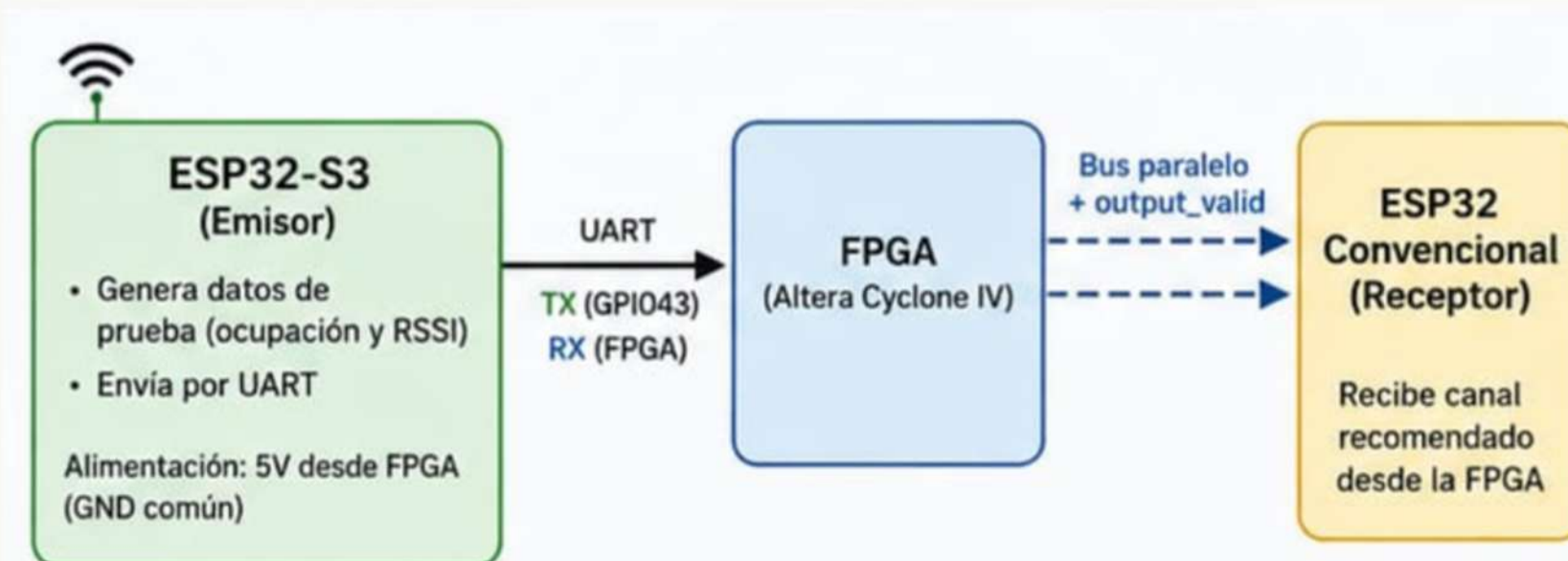
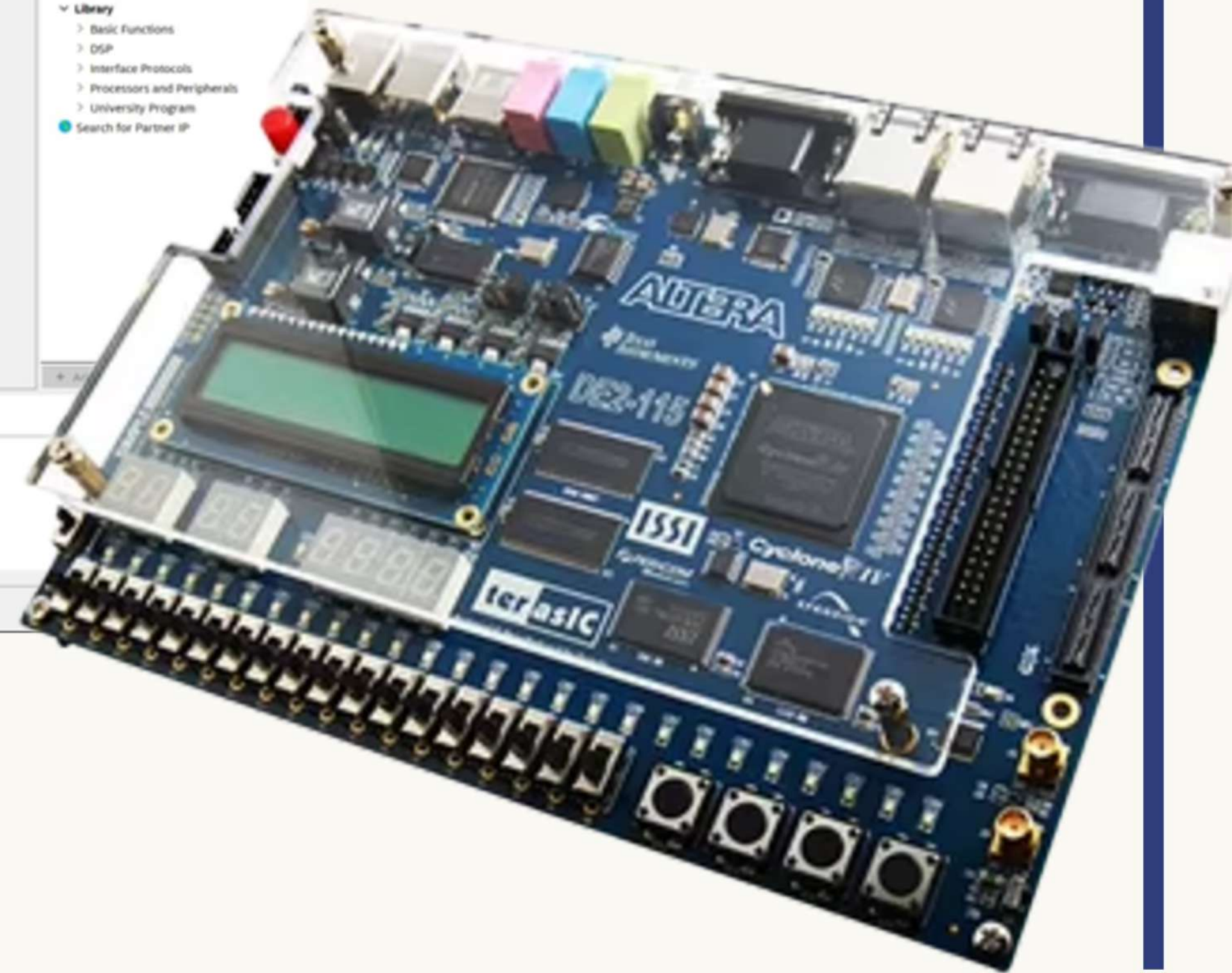
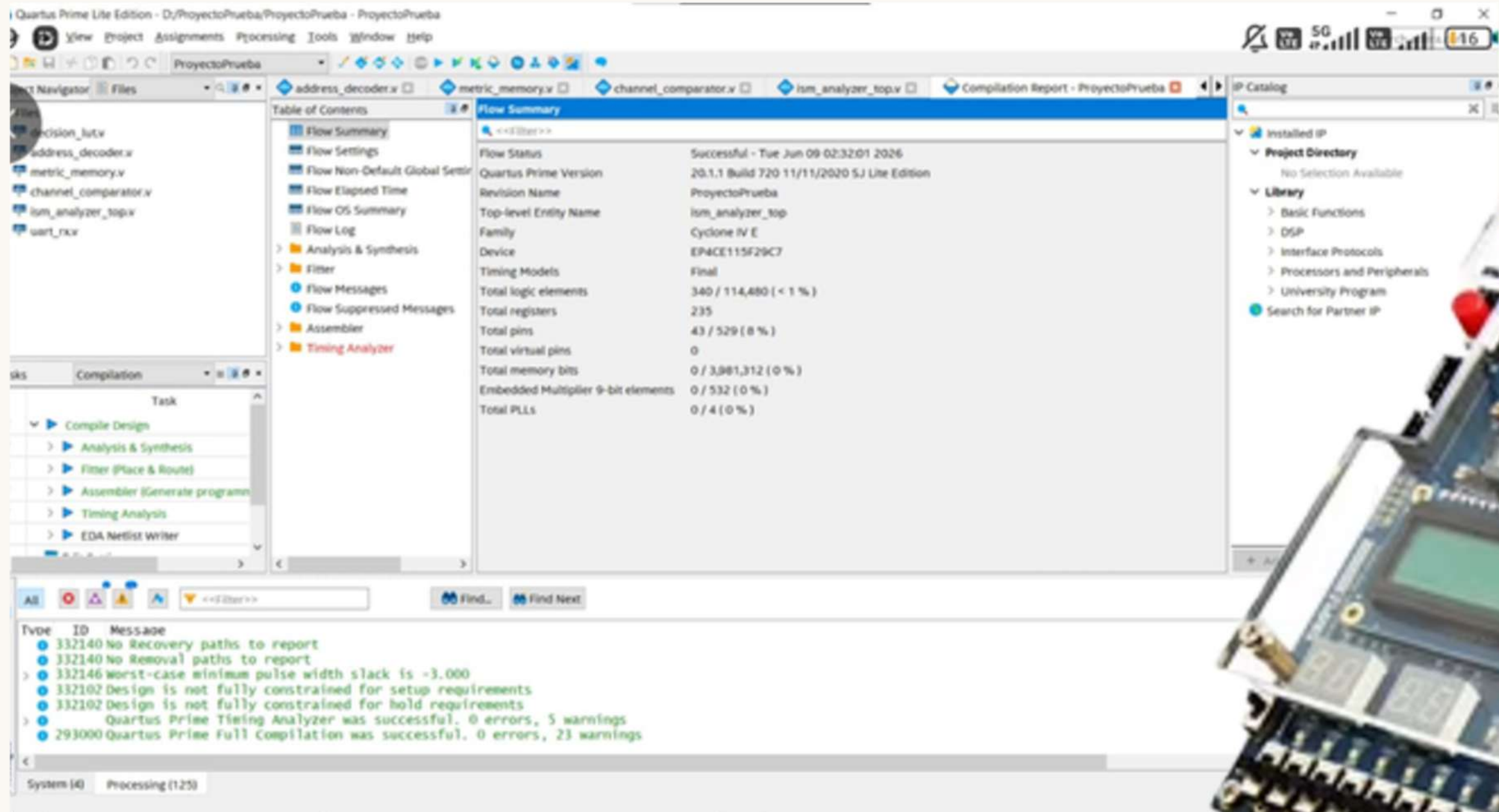
Recursos software

El desarrollo emplea bibliotecas estándar del ecosistema Arduino y ESP32, incluyendo:

RECURSO	FUNCIÓN
Arduino.h	Funciones básicas del framework Arduino
WiFi.h	Escaneo y administración de redes WiFi
Serial	Comunicación serie con la computadora
Serial2	Comunicación UART con la FPGA
digitalRead()	Lectura de entradas digitales
attachInterrupt()	Gestión de interrupciones
WiFi.scanNetworks()	Escaneo de redes inalámbricas
WiFi.softAP()	Creación de punto de acceso WiFi



Los módulos ESP32 funcionan como interfaces de adquisición y comunicación, mientras que el procesamiento principal se realiza en la FPGA. La arquitectura distribuye eficientemente las tareas y mantiene un bajo consumo de recursos de memoria y almacenamiento.

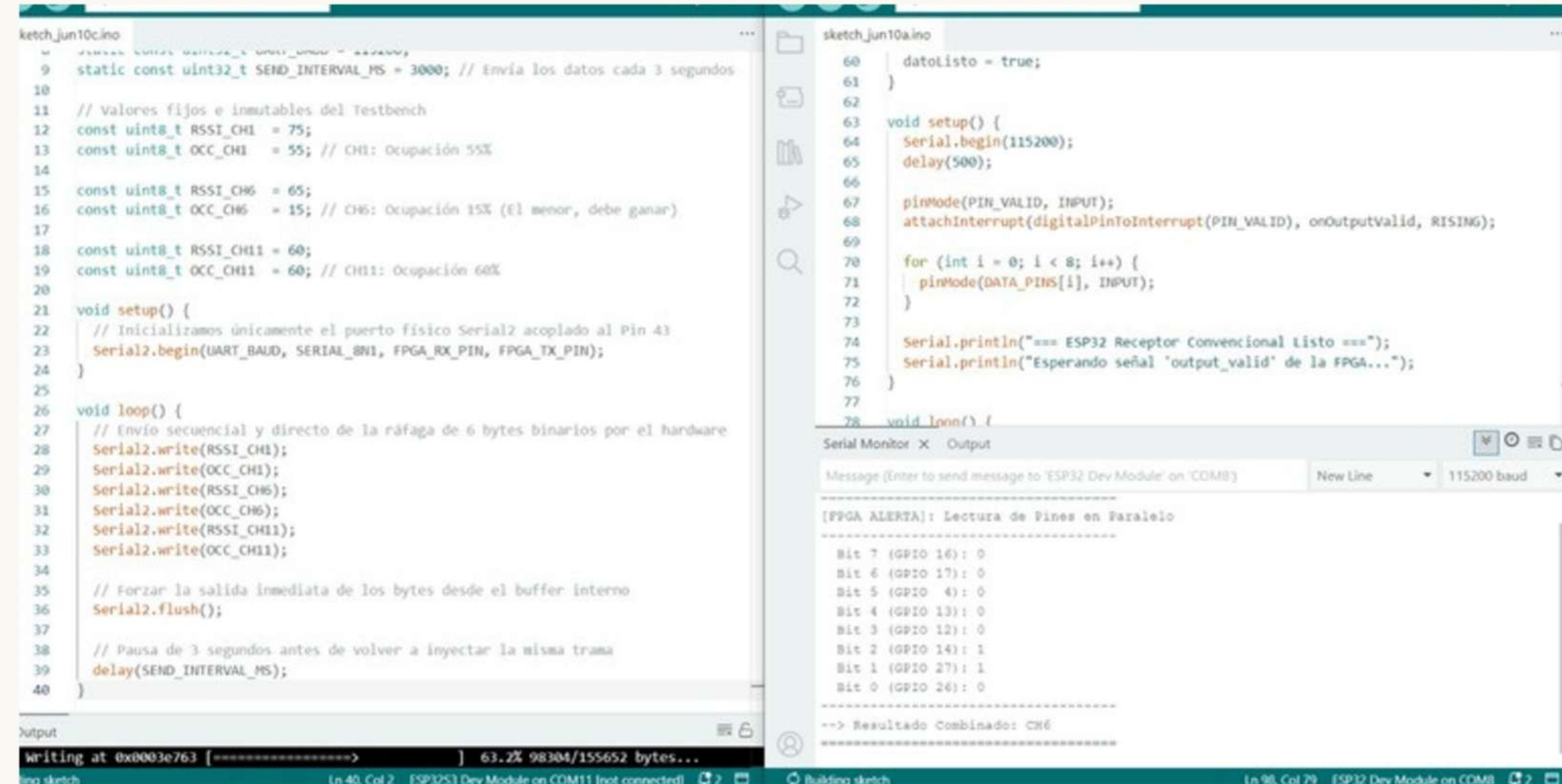


MEDICIÓN DE MÉTRICAS INICIALES

Los módulos ESP32 funcionan como interfaces de adquisición y comunicación, mientras que el procesamiento principal se realiza en la FPGA. La arquitectura distribuye eficientemente las tareas y mantiene un bajo consumo de recursos de memoria y almacenamiento.

Escenario de prueba

CANAL	RSSI (%)	OCUPACIÓN (%)
Canal 1	75	55
Canal 6	65	15
Canal 11	60	60



The screenshot displays the Arduino IDE interface. On the left, a sketch named 'sketch_jun10c.ino' contains C++ code for an ESP32. It defines constants for RSSI and occupancy values for three channels (CH1, CH6, CH11) and includes a loop that writes these values to a serial port. On the right, another sketch named 'sketch_jun10a.ino' is shown, which sets up a serial port and configures GPIO pins. Below the sketches, the 'Serial Monitor' window is open, showing the output of the first sketch. The output indicates that the ESP32 is ready and is waiting for a signal from the FPGA. It then displays a list of GPIO pin states for bits 7 through 0, with bit 1 and bit 2 being high (1) and the others being low (0). The final line of output is 'Resultado Combinado: CH6'.

```
sketch_jun10c.ino
9 static const uint32_t SEND_INTERVAL_MS = 3000; // Envía los datos cada 3 segundos
10
11 // Valores fijos e inmutables del Testbench
12 const uint8_t RSSI_CH1 = 75;
13 const uint8_t OCC_CH1 = 55; // CH1: Ocupación 55%
14
15 const uint8_t RSSI_CH6 = 65;
16 const uint8_t OCC_CH6 = 15; // CH6: Ocupación 15% (El menor, debe ganar)
17
18 const uint8_t RSSI_CH11 = 60;
19 const uint8_t OCC_CH11 = 60; // CH11: Ocupación 60%
20
21 void setup() {
22   // Inicializamos únicamente el puerto físico Serial2 acoplado al Pin 43
23   Serial2.begin(UART_BAUD, SERIAL_BN1, FPGA_RX_PIN, FPGA_TX_PIN);
24 }
25
26 void loop() {
27   // Envío secuencial y directo de la ráfaga de 6 bytes binarios por el hardware
28   Serial2.write(RSSI_CH1);
29   Serial2.write(OCC_CH1);
30   Serial2.write(RSSI_CH6);
31   Serial2.write(OCC_CH6);
32   Serial2.write(RSSI_CH11);
33   Serial2.write(OCC_CH11);
34
35   // Forzar la salida inmediata de los bytes desde el buffer interno
36   Serial2.flush();
37
38   // Pausa de 3 segundos antes de volver a inyectar la misma trama
39   delay(SEND_INTERVAL_MS);
40 }

sketch_jun10a.ino
60 datoListo = true;
61 }
62
63 void setup() {
64   Serial.begin(115200);
65   delay(500);
66
67   pinMode(PIN_VALID, INPUT);
68   attachInterrupt(digitalPinToInterrupt(PIN_VALID), onOutputValid, RISING);
69
70   for (int i = 0; i < 8; i++) {
71     pinMode(DATA_PINS[i], INPUT);
72   }
73
74   Serial.println("=== ESP32 Receptor Convencional listo ===");
75   Serial.println("Esperando señal 'output_valid' de la FPGA...");
76 }
77
78 void loop() {

Serial Monitor X Output
Message (Enter to send message to 'ESP32 Dev Module' on 'COM8') New Line 115200 baud

[FPGA ALERTA]: Lectura de Pines en Paralelo
-----
Bit 7 (GPIO 16): 0
Bit 6 (GPIO 17): 0
Bit 5 (GPIO 4): 0
Bit 4 (GPIO 13): 0
Bit 3 (GPIO 12): 0
Bit 2 (GPIO 14): 1
Bit 1 (GPIO 27): 1
Bit 0 (GPIO 26): 0
-----
--> Resultado Combinado: CH6
-----

Output
writing at 0x0003e763 [=====] 63.2% 98304/155652 bytes...
In 40, Col 2 ESP32 Dev Module on COM11 [not connected] Building sketch In 98, Col 79 ESP32 Dev Module on COM8
```

Resultado esperado

- Canal recomendado = CH6

Porque presenta el menor nivel de ocupación.

MEDICIÓN DE MÉTRICAS INICIALES

Parámetros del sistema

PARÁMETRO	VALOR
Tiempo por bit UART	8.68 μ s
Tiempo por byte UART	86.8 μ s
Número de bytes transmitidos	6
Tiempo total de transmisión	520.8 μ s
Pausa agregada por testbench	120 μ s
Tiempo de procesamiento FPGA	0.24 μ s
Tiempo total estimado	641.04 μ s

Resultados Simulados vs Reales

PARÁMETRO	SIMULACIÓN	HARDWARE REAL
Canal recomendado	CH6	CH6
Valor binario obtenido	110	110
Valor decimal obtenido	6	6
Señal output_valid	Activa	Activa
Resultado esperado	Correcto	Correcto
Tiempo de respuesta aproximado	641.04 μ s	520.8 μ s + procesamiento

Análisis y contraste

- Simulación y hardware obtuvieron el mismo resultado (Canal 6).
- La simulación presentó mayor latencia debido a retardos añadidos en el testbench.
- La FPGA opera a 50 MHz, muy por debajo de su Fmax de 157.28 MHz, garantizando un funcionamiento estable.
- La baja utilización de recursos permite futuras ampliaciones del sistema sin afectar el rendimiento.

ESP32 - WROOM32

```
esptool v3.2.0
Serial port COM12:
Connecting....
Connected to ESP32 on COM12:
Chip type:      ESP32-D0WD-V3 (revision v3.1)
Features:       Wi-Fi, BT, Dual Core + LP Core, 240MHz,
                Vref calibration in eFuse, Coding Scheme None
Crystal frequency: 40MHz
MAC:            08:b6:1f:b8:bc:2c
Uploading stub flasher...
Running stub flasher...
Stub flasher running.
Changing baud rate to 921600...
Changed.

Configuring flash size...
Flash will be erased from 0x00001000 to 0x00007fff...
Compressed 25024 bytes to 16018...
Writing at 0x00001000 [          ] 0.0% 0/16018 bytes...
Writing at 0x000071c0 [=====] 100.0% 16018/16018 bytes...
Wrote 25024 bytes (16018 compressed) at 0x00001000 in 0.6 seconds (322.9 kbit/s).
Verifying written data...
Hash of data verified.

Flash will be erased from 0x00008000 to 0x00008fff...
Compressed 3072 bytes to 146...
Writing at 0x00008000 [          ] 0.0% 0/146 bytes...
Writing at 0x00008c00 [=====] 100.0% 146/146 bytes...
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.1 seconds (194.6 kbit/s).
Verifying written data...
Hash of data verified.

Flash will be erased from 0x0000e000 to 0x0000ffff...
Compressed 8192 bytes to 47...
Writing at 0x0000e000 [          ] 0.0% 0/47 bytes...
Writing at 0x00010000 [=====] 100.0% 47/47 bytes...
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (520.2 kbit/s).
Verifying written data...
Hash of data verified.

Flash will be erased from 0x00010000 to 0x00056fff...
Compressed 287648 bytes to 158126...
Writing at 0x00010000 [          ] 0.0% 0/158126 bytes...
Writing at 0x0001bc45 [==>      ] 10.4% 16384/158126 bytes...
Writing at 0x00028fef [=====>   ] 20.7% 32768/158126 bytes...
Writing at 0x0002c675 [=====>   ] 31.1% 49152/158126 bytes...
Writing at 0x000341f0 [=====>   ] 41.4% 65536/158126 bytes...
Writing at 0x0003954e [=====>   ] 51.8% 81920/158126 bytes...
Writing at 0x0003e9c3 [=====>   ] 62.2% 98304/158126 bytes...
Writing at 0x0004413d [=====>   ] 72.5% 114688/158126 bytes...
Writing at 0x0004a83b [=====>   ] 82.9% 131072/158126 bytes...
Writing at 0x00052664 [=====>   ] 93.3% 147456/158126 bytes...
Writing at 0x000563a0 [=====] 100.0% 158126/158126 bytes...
Wrote 287648 bytes (158126 compressed) at 0x00010000 in 3.1 seconds (747.6 kbit/s).
Verifying written data...
Hash of data verified.
Hard resetting via RTS pin...
```

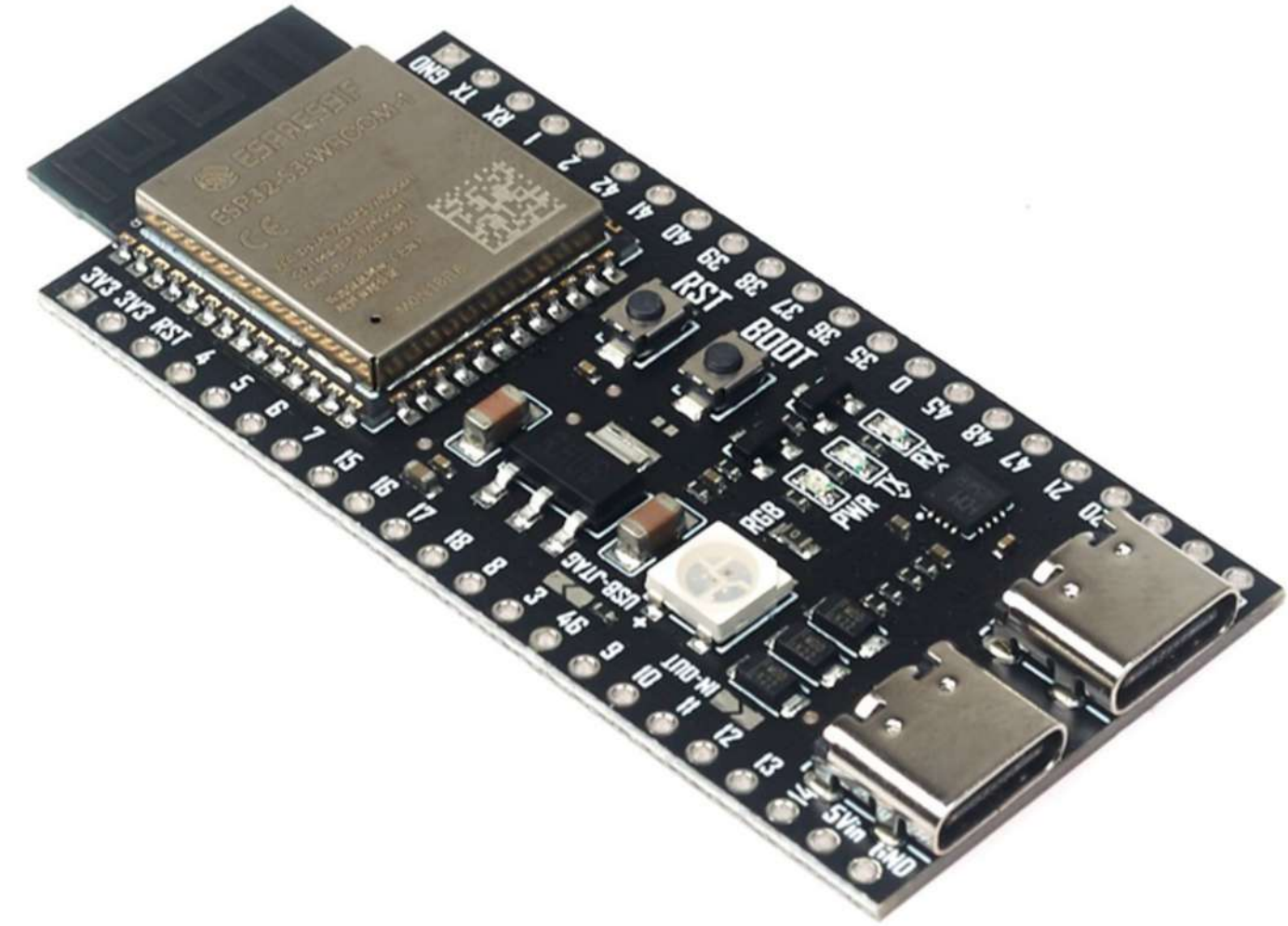


ESP32-S3 Dev Kit

```
Monitor Serie x
Mensaje (Intro para enviar el mensaje a 'ESP32S3 Dev Module' en 'COM7') Nueva línea 115200 baudio

esptool v5.2.0
Serial port COM7:
Connecting....
Connected to ESP32-S3 on COM7:
Chip type:      ESP32-S3 (QFN56) (revision v0.2)
Features:       Wi-Fi, BT 5 (LE), Dual Core + LP Core, 240MHz,
                Embedded PSRAM 8MB (AP_3v3)
Crystal frequency: 40MHz
MAC:            44:1b:f6:ff:32:28
Uploading stub flasher...
Running stub flasher...
Stub flasher running.
Changing baud rate to 921600...
Changed.

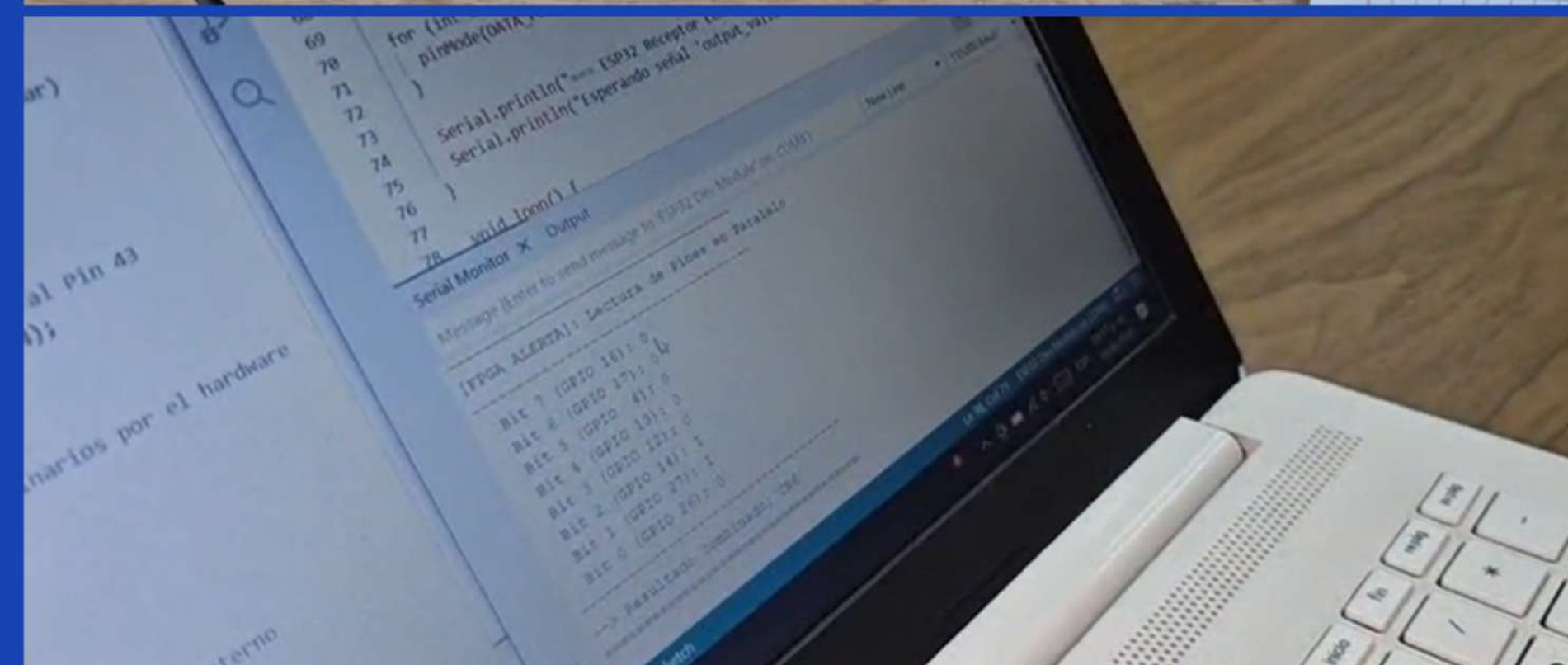
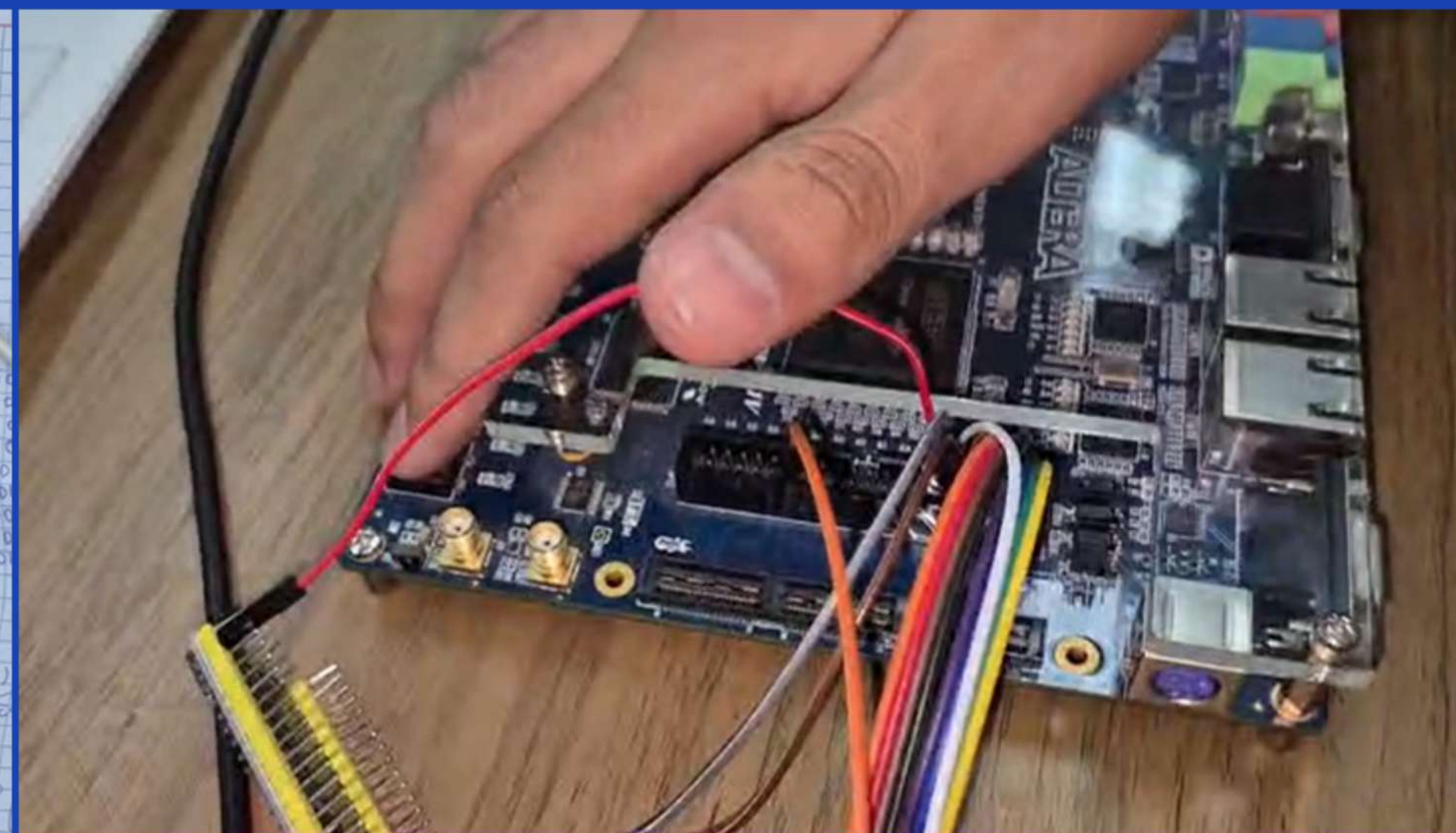
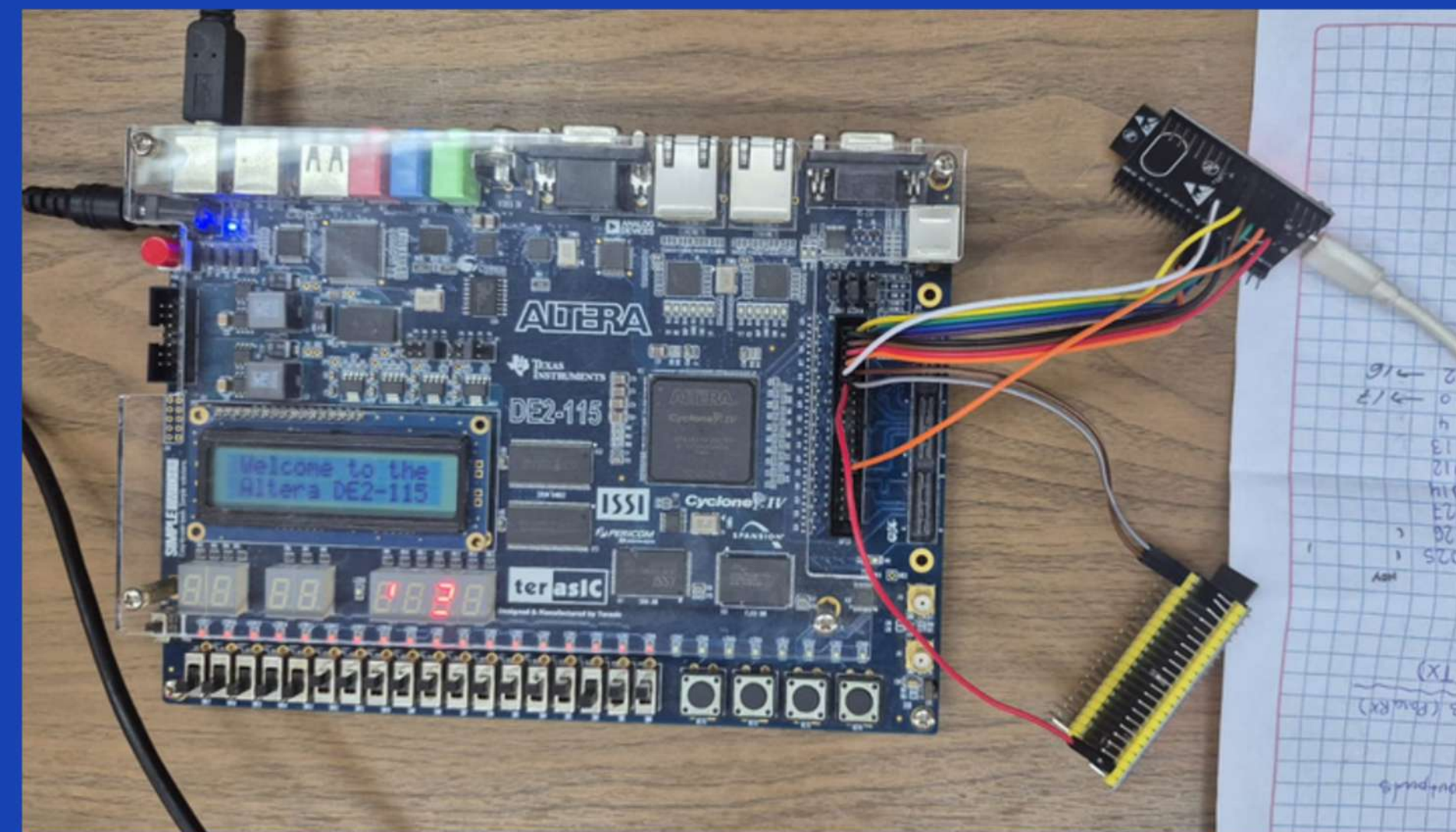
Configuring flash size...
Flash will be erased from 0x00000000 to 0x00004fff...
Compressed 19984 bytes to 13014...
Writing at 0x00000000 [=====] 100.0% 13014/13014 bytes...
Wrote 19984 bytes (13014 compressed) at 0x00000000 in 0.3 seconds (489.8 kbit/s).
Verifying written data...
Hash of data verified.
Flash will be erased from 0x00008000 to 0x00008fff...
Compressed 3072 bytes to 146...
Writing at 0x00008000 [=====] 100.0% 146/146 bytes...
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (697.5 kbit/s).
Verifying written data...
Hash of data verified.
Flash will be erased from 0x0000e000 to 0x0000ffff...
Compressed 8192 bytes to 47...
Writing at 0x0000e000 [=====] 100.0% 47/47 bytes...
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (925.7 kbit/s).
Verifying written data...
Hash of data verified.
Flash will be erased from 0x00010000 to 0x00055fff...
Compressed 285504 bytes to 155755...
Writing at 0x00010000 [====>] 0.0% 0/155755 bytes...
Writing at 0x0001ba6d [====>] 10.5% 16384/155755 bytes...
Writing at 0x000280a9 [====>] 21.0% 32768/155755 bytes...
Writing at 0x0002daef [====>] 31.6% 49152/155755 bytes...
Writing at 0x00033988 [====>] 42.1% 65536/155755 bytes...
Writing at 0x00038f91 [====>] 52.6% 81920/155755 bytes...
Writing at 0x0003e720 [====>] 63.1% 98304/155755 bytes...
Writing at 0x00044151 [====>] 73.6% 114688/155755 bytes...
Writing at 0x0004bb5e [====>] 84.2% 131072/155755 bytes...
Writing at 0x000528f4 [====>] 94.7% 147456/155755 bytes...
Writing at 0x00055b40 [====>] 100.0% 155755/155755 bytes...
Wrote 285504 bytes (155755 compressed) at 0x00010000 in 2.4 seconds (934.0 kbit/s).
Verifying written data...
Hash of data verified.
Hard resetting via RTS pin...
```



EVIDENCIA DE FUNCIONAMIENTO

<https://youtu.be/ZwhxzbEeRf4>

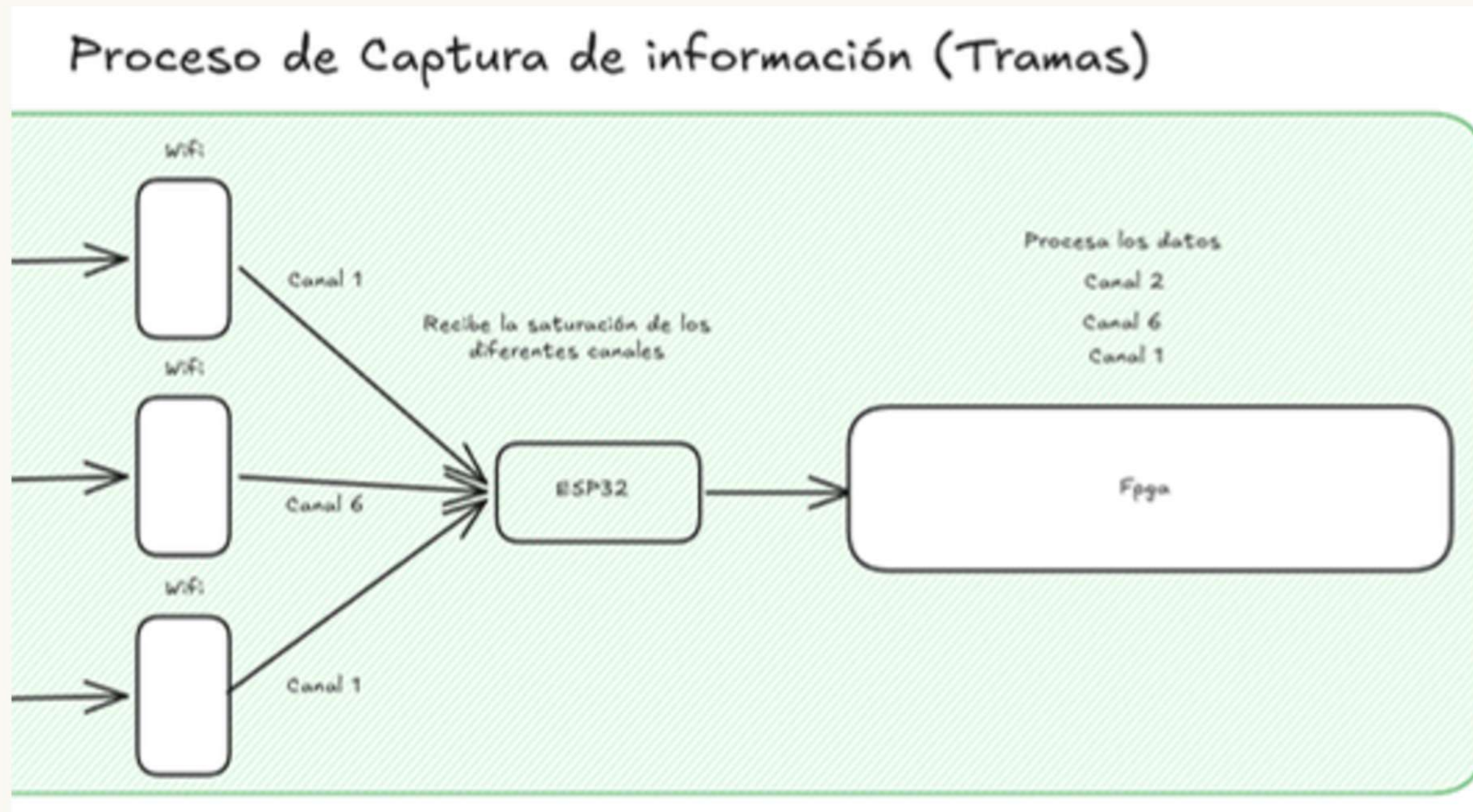
Sección 1: Prototipo físico ensamblado y funcional



Valor agregado

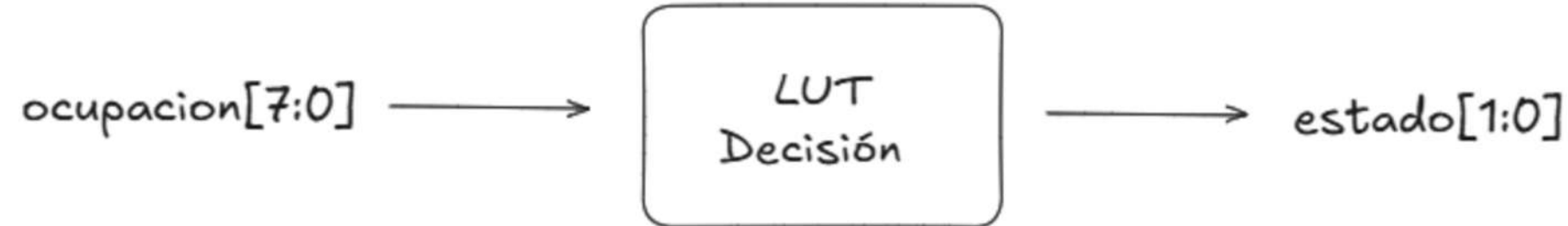
El uso de una FPGA permite implementar el procesamiento y la toma de decisiones directamente en hardware digital, logrando una ejecución paralela y determinística con baja latencia

↪ A diferencia de un microcontrolador convencional, la FPGA puede controlar simultáneamente módulos de memoria, decodificación de direcciones, clasificación LUT y comparación de canales mediante una FSM dedicada



CÓDIGO Y PSEUDOCÓDIGO

1. Módulo LUT para la decisión



Codificación de salida:

00 = Libre (0 % – 30 %)
01 = Moderado (31 % – 70 %)
10 = Congestionado (71 % – 100 %)
11 = Reservado (no utilizado)

Pseudocódigo

MÓDULO: decision_lut

ENTRADA: ocupacion [7:0]

SALIDA: estado [1:0]

PROCESO COMBINACIONAL:

Si ocupacion ≤ 30 → estado = 00 (Libre)

Si ocupacion ≤ 70 → estado = 01 (Moderado)

Si ocupacion > 70 → estado = 10 (Congestionado)

Módulo decodificador de direcciones



Pseudocódigo

MÓDULO: address_decoder

ENTRADA: addr [3:0]

SALIDAS: sel_rssi_ch1, sel_occ_ch1, sel_state_ch1,
sel_rssi_ch6, sel_occ_ch6, sel_state_ch6,
sel_rssi_ch11, sel_occ_ch11, sel_state_ch11,
sel_recommended

PROCESO COMBINACIONAL:

Poner todas las señales de selección en 0

Según el valor de addr:

0x0 → sel_rssi_ch1 = 1

0x1 → sel_occ_ch1 = 1

0x2 → sel_state_ch1 = 1

0x4 → sel_rssi_ch6 = 1

0x5 → sel_occ_ch6 = 1

0x6 → sel_state_ch6 = 1

0x8 → sel_rssi_ch11 = 1

0x9 → sel_occ_ch11 = 1

0xA → sel_state_ch11 = 1

0xF → sel_recommended = 1

Otro → todas las señales permanecen en 0

Módulo memoria de métricas



Escritura: síncrona (flanco ↑ de clk, cuando write_enable = 1)
Lectura: asíncrona (data_out = mem[addr] siempre)

Pseudocódigo

MÓDULO: metric_memory

ENTRADAS: clk, rst, write_enable, addr [3:0], data_in [7:0]

SALIDA: data_out [7:0]

DEFINICIÓN: mem[0..15] — arreglo de 16 registros de 8 bits

PROCESO SÍNCRONO (en flanco positivo de clk):

Si rst = 1:

Para i = 0 hasta 15: mem[i] ← 0

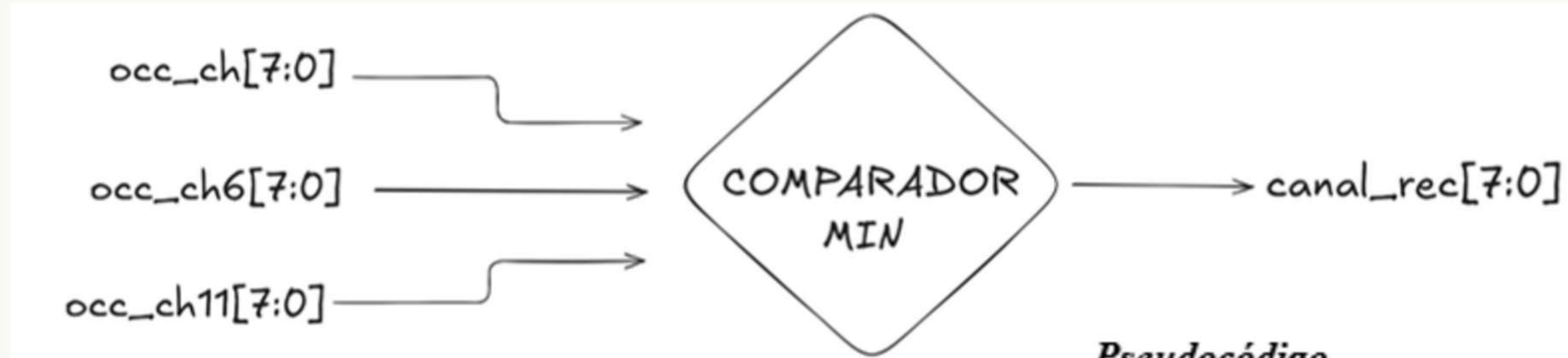
Si no y write_enable = 1:

mem[addr] ← data_in

PROCESO COMBINACIONAL (lectura asíncrona):

data_out = mem[addr]

Módulo comparador de canales



Pseudocódigo

MÓDULO: channel_comparator

ENTRADAS: occ_ch1 [7:0], occ_ch6 [7:0], occ_ch11 [7:0]

SALIDA: canal_recomendado [7:0]

PROCESO COMBINACIONAL:

Si $\text{occ_ch1} \leq \text{occ_ch6}$ Y $\text{occ_ch1} \leq \text{occ_ch11}$:

canal_recomendado $\leftarrow$ 1

Si no, si $\text{occ_ch6} < \text{occ_ch1}$ Y $\text{occ_ch6} \leq \text{occ_ch11}$:

canal_recomendado $\leftarrow$ 6

Si no:

canal_recomendado $\leftarrow$ 11

(En caso de empate se prioriza: canal 1 > canal 6 > canal 11)

Simulación

The screenshot displays the Quartus Prime Lite Edition interface with the 'Compilation Report - ISM_ANALYZER' open. The report shows the following table:

Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time (on all processors)
1 Analysis_inthesis	00:00:09	1.0	4857 MB	00:00:21
2 Filter	00:00:21	1.0	6719 MB	00:00:36
3 Assembler	00:00:03	1.0	4702 MB	00:00:53
4 Timing Analyzer	00:00:02	1.1	5007 MB	00:00:02
5 EDA Netlist Writer	00:00:00	1.0	4661 MB	00:00:00
6 Total	00:00:35	--	--	00:01:02

The 'Messages' window at the bottom shows the following log:

```
Running Quartus Prime Analysis & Synthesis
Command: quartus_map --read_settings_files-on --write_settings_files-off ISM_ANALYZER -c ISM_ANALYZER
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
20030 Parallel compilation is enabled and will use 16 of the 16 processors detected
12021 Found 1 design units, including 1 entities, in source file address_decoder.v
12021 Found 1 design units, including 1 entities, in source file decision_lut.v
12021 Found 1 design units, including 1 entities, in source file metric_memory.v
12021 Found 1 design units, including 1 entities, in source file channel_comparator.v
12021 Found 1 design units, including 1 entities, in source file ism_analyzer_top.v
12127 Elaborating entity "ism_analyzer_top" for the top level hierarchy
12128 Elaborating entity "decision_lut" for hierarchy "decision_lut::LUT_Cnx1"
12128 Elaborating entity "channel_comparator" for hierarchy "channel_comparator::CMP"
12128 Elaborating entity "metric_memory" for hierarchy "metric_memory::MPX"
10240 Verilog HDL Always Construct warning at metric_memory.v(15): Inferring latch(es) for variable "t1", which holds its previous value in one or more paths through the always construct
```

The screenshot displays the Quartus Prime Lite Edition interface with the 'Compilation Report - ISM_ANALYZER' open. The report shows the following table:

Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time (on all processors)
1 Analysis_inthesis	00:00:09	1.0	4857 MB	00:00:21
2 Filter	00:00:21	1.0	6719 MB	00:00:36
3 Assembler	00:00:03	1.0	4702 MB	00:00:53
4 Timing Analyzer	00:00:02	1.1	5007 MB	00:00:02
5 EDA Netlist Writer	00:00:00	1.0	4661 MB	00:00:00
6 Total	00:00:35	--	--	00:01:02

The 'Messages' window at the bottom shows the following log:

```
Running Quartus Prime Analysis & Synthesis
Command: quartus_map --read_settings_files-on --write_settings_files-off ISM_ANALYZER -c ISM_ANALYZER
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
20030 Parallel compilation is enabled and will use 16 of the 16 processors detected
12021 Found 1 design units, including 1 entities, in source file address_decoder.v
12021 Found 1 design units, including 1 entities, in source file decision_lut.v
12021 Found 1 design units, including 1 entities, in source file metric_memory.v
12021 Found 1 design units, including 1 entities, in source file channel_comparator.v
12021 Found 1 design units, including 1 entities, in source file ism_analyzer_top.v
12127 Elaborating entity "ism_analyzer_top" for the top level hierarchy
12128 Elaborating entity "decision_lut" for hierarchy "decision_lut::LUT_Cnx1"
12128 Elaborating entity "channel_comparator" for hierarchy "channel_comparator::CMP"
12128 Elaborating entity "metric_memory" for hierarchy "metric_memory::MPX"
10240 Verilog HDL Always Construct warning at metric_memory.v(15): Inferring latch(es) for variable "t1", which holds its previous value in one or more paths through the always construct
```

A system monitor window is overlaid on the right side of the screen, showing the following performance metrics:

- CPU: 17% 3.69 GHz
- Memoria: 13.3/15.4 GB (86%)
- Disco 0 (G: C): 150 (MB/s)
- Wi-Fi: 6.37 Kbps
- GPU 0: AMD Radeon (TM) 7 5825U with Radeon Graphics
- Velocidad de base: 2.00 GHz
- Uso: 17%
- Procesos: 322
- Subprocesos: 5332
- Identificadores: 170625
- Procesadores lógicos: 16
- Virtualización: Habilitado
- Caché L1: 512 KB
- Caché L2: 4.0 MB
- Caché L3: 16.0 MB